EMBEDDED CONTROL SOFTWARE

Steering Turret Control System

Engineering documentation

Low-level embedded Linux control for a military steering turret, with real-time scheduling, CAN motor integration, and fault handling for critical defense applications.

Dimitrios Asimakopoulos

Original technical report: May 2026

Portfolio edition: October 2026

C11 / C++17 | Linux / ARM64 | SocketCAN | TCP / UDP | Qt6

This edition focuses on software architecture, implementation, and verification. Company branding, personnel references, internal requirement traceability, and internal document references have been removed.

About this edition

STC connects a control application to a multi-axis motor system. It validates incoming commands, dispatches motion through a dedicated worker, and publishes position, velocity, state, and health information. A common backend interface supports both CAN hardware and an in-process simulator.

The software combines a C runtime and binary protocol codec, C++ simulation, Nanotec CANopen / CiA 402 integration, a separate MYACTUATOR CAN driver, and Qt diagnostic tools. Linux scheduling policies, capabilities, monotonic clocks, and systemd deployment are part of the integration work.

How to read the report

The detailed technical chapters retain their original section and printed page numbers. Numbering gaps are intentional: the internal requirements chapter is omitted. The contents page and PDF bookmarks below point to the retained material.

This is an edited snapshot of the supplied technical report. For current file paths and runnable build commands, use README.md and manuals/SETUP.md in the repository. Configuration excerpts are examples and must be reviewed for the actual bench.

Verification boundary

An 18-axis software model does not establish a full 18-axis hardware qualification. Simulation validates functional behavior, not physical motor response or bus timing. Real-time scheduling settings do not establish certified hard real-time deadlines or a certified safety level.

Laser fields describe commanded software state; a physical laser backend is not integrated. Autonomous target tracking and payload operation are outside the demonstrated software scope.

Contents

1	Introduction	1
2	System context	8
4	Architecture overview	39
5	Operating system and real-time environment	43
6	Communication protocol	52
7	Server runtime	60
8	Motion control: CANopen, CiA 402, vendor CAN	77
9	Simulation, user interfaces, and verification	94
	References	105
A	ICD byte-level reference	113
B	State machine reference	127
C	Configuration schema and examples	136
D	CANopen and CiA 402 mapping	155

Page numbers refer to the printed technical-body pages. The original section numbering is retained for cross-references. Internal requirement tables and hardware payload-design variants are omitted from this portfolio edition.

Chapter 1

Introduction

1.1 Project Purpose and System Scope

The Steering Turret Control System (STC) is the software subsystem responsible for commanding, supervising, and monitoring the controlled functions of the steering turret platform. From the software point of view, the turret is a networked multi-axis electromechanical system that must receive external commands, execute bounded motion, supervise health and faults, and continuously report operational state and telemetry.

This document is written as the authoritative technical description of that control stack. Its purpose is to explain the system as an engineering whole: what the turret control software is, how it is structured, which interfaces and hardware it depends on, how it behaves at runtime, and how it should be built, tested, deployed, and maintained. The document is therefore aimed not only at software developers, but also at integration engineers, testers, maintainers, and technical reviewers.

Within this scope, the software-visible turret includes the controlled turret axes, camera motion, SUT and mirror actuation, laser control, telemetry generation, state supervision, diagnostics, and the external Control Unit interface. The focus of the document is the control and software architecture that makes these functions deterministic, reproducible, and operationally understandable.

1.2 STC Responsibilities

Within the overall system, STC is responsible for the control-plane and data-plane functions that connect the external Control Unit to the local turret hardware and local turret measurements back to the external system.

More specifically, STC is responsible for:

- receiving commands from the external Control Unit through the defined software interface
- validating commands against protocol rules, current state, axis mappings, and motion limits
- translating accepted commands into internal actions and low-level device operations
- maintaining the turret operational state machine, including initialization, safe mode, operational mode, self-levelling, test flows, storage mode, and fail behavior
- managing command responses and telemetry transmission
- supervising motor status, watchdogs, communication health, and software diagnostics
- enforcing software-side safe behavior when faults, invalid transitions, stale communications, or prohibited actions occur
- providing the abstraction boundary between the external ICD and the device-specific low-level motor libraries.

In the current project structure, this responsibility is implemented through a central server runtime and a layered motion subsystem. The higher layer owns protocol handling, state logic, shared data, and diagnostics, while the lower layer interfaces with the specific actuator families used by the turret. This includes the Nanotec motor path based on CANopen and CiA 402 / DS402, as well as the MYACTUATOR path based on the vendor CAN protocol and its dedicated low-level library.

For clarity, several surrounding topics are intentionally outside the scope of this software document except where they directly affect control behavior: external Control Unit UI design beyond the ICD, mechanical design beyond limits, scaling, mappings, and kinematic meaning, higher-level mission logic beyond the STC boundary, and full system safety or certification cases be-

yond the implemented diagnostics, fault handling, and software gating behavior.

1.3 Architectural Motivation

The architecture of STC exists because the turret control problem is inherently deterministic, stateful, and hardware-facing. The software must accept external commands over a defined network protocol, transform them into predictable motion or lifecycle actions, preserve control authority and safety rules, and publish telemetry in a form that is stable and externally consumable.

Three engineering motivations dominate the design.

First, motion execution must be deterministic. A turret command is an operation that must respect state gating, configured limits, axis mappings, communication health, and the semantics of the underlying motor protocol. For that reason, motion execution is separated from command parsing and isolated behind dedicated low-level libraries and a motion backend abstraction.

Second, command and telemetry exchange are first-class functions of the system. The Control Unit does not interact with the turret through ad hoc calls or local APIs, but through a formal ICD that defines message identity, payload semantics, sequencing, and telemetry structure. The architecture therefore treats networking, parsing, packing, acknowledgement, and telemetry publication as core runtime behavior.

Third, the runtime must remain robust on an embedded Linux platform. The system combines multithreading, shared state, real-time oriented timing behavior, raw CAN communication, and hardware supervision. The software structure is therefore designed to keep fast protocol handling, motion execution, and telemetry publication separated enough to remain understandable, testable, and operationally stable.

1.4 Current Software Stack

The current STC software stack is centered on a CMake-based build system that defines a mixed-language codebase: the core runtime is written in C11, while the simulator and optional desktop tooling use C++17. The main executable is `stc_server`; companion tools include a TCP command injector,

a UDP telemetry receiver, a Qt-based diagnostic GUI, and multiple unit and integration tests. The build also supports a simulation path through a dedicated simulator library and backend selection at compile time.

At code level, the runtime stack is organized around the following major modules:

- `stc_server`: the main multithreaded daemon that owns the TCP command path, shared runtime state, telemetry publication, and motion-action orchestration
- `stc_icd`: a transport-agnostic binary codec for TCP command/response frames and UDP telemetry packets, with explicit little-endian manual packing and unpacking
- `stc_model`, `stc_config`, `stc_actions`, `stc_limits`: the core domain model, axis configuration, command normalization, and motion-limit logic
- `stc_state`, `stc_faults`, `stc_diag`: the operational state machine, fault handling, and diagnostic-code generation;
- `motion_backend`: the abstraction that decouples the server from the concrete motion implementation, allowing either real CAN-connected hardware or the simulator backend
- `pd4c`: the low-level Nanotec CANopen library, designed as a single-threaded SocketCAN component that uses PDOs for runtime control and SDOs for setup and diagnostics
- `x8_25`: the low-level MYACTUATOR library for the X8/L actuator family, implemented as a bounded, no-heap, callback-driven CAN library with both fast send-only control functions and synchronous request/reply helpers.

The target operating environment is an embedded Linux system on the RockPro64 platform. The runtime uses standard POSIX and Linux interfaces, including TCP/IP and UDP sockets, `pthread` threading, monotonic timing APIs, Linux scheduling controls, and `PF_CAN`/`CAN_RAW` SocketCAN communication. On the control side, the stack uses two different motor-protocol families: CANopen with CiA 402 / DS402 semantics for the Nanotec path, and the vendor-specific MYACTUATOR CAN protocol for the X8/L path. The repository also includes a dedicated SocketCAN test application for the MYACTUATOR library that exercises identity reads, status acquisition, active replies, torque control, speed control, absolute and incremental position

control, and motion-mode commands.

From an interface perspective, the software stack is built around a deterministic binary ICD over Ethernet, with TCP used for command/response exchange and UDP used for telemetry publication. At the runtime level, the server structure is explicitly multithreaded: command reception and fast validation occur in the network path, motion activity is delegated to the motion layer, and telemetry is published as a separate activity. This separation is fundamental to both determinism and maintainability.

1.5 Reader Map

This document is organized so that the reader first understands the system boundary and requirements, then the runtime platform and interfaces, and only afterwards the implementation details of the server, motion stack, verification assets, and reference appendices.

- **Chapter 2 – System Context** defines the end-to-end control path from the external Control Unit to the STC daemon, motion backend, CAN-based device network, drives, actuators, and telemetry outputs. It also introduces the turret domain model, including the canonical 18-axis configuration, SUTs, mirrors, lasers, and functional mappings.

Chapter 3 is omitted in this edition. The retained chapters describe the software architecture, runtime, motor interfaces, and verification. Internal requirement identifiers and traceability tables are not included.

- **Chapter 4 – Architecture Overview** presents the repository structure, build products, runtime modules, and architectural separation between core logic, motion abstraction, simulator support, hardware backend, tools, and tests.
- **Chapter 5 – Operating System and Real-Time Environment** documents the RockPro64 embedded Linux baseline, SocketCAN model, scheduling policy, real-time startup behavior, systemd deployment approach, capability requirements, CPU-affinity strategy, and operational checks needed for reproducible execution.
- **Chapter 6 – Communication Protocol** defines the CU–STC software interface at semantic level. It explains the TCP command and response model, UDP telemetry streams, protocol encoding rules, com-

mand payload behavior, response philosophy, telemetry destination selection, and ICD tooling.

- **Chapter 7 – Server Runtime** describes the internal behavior of `stc_server`. It covers the thread model, shared-state design, action queue, command processing path, state gating, fault detection, degraded behavior, safe stop handling, backend ownership, stream watchdogs, BIT runtime semantics, observability, and runtime invariants.
- **Chapter 8 – Motion Control over CANopen, DS402, and MYACTUATOR CAN** documents the actuator-control layer. It explains the Nanotec PD4/PD6 CANopen and CiA 402 path, PDO and SDO usage, the `pd4c` low-level library, the MYACTUATOR CAN protocol and library, axis-level conversions, clamping, scaling, commissioning procedures, and motion safety rules.
- **Chapter 9 – Simulation, User Interfaces, and Verification** documents the simulator backend, Qt diagnostic GUI, CLI tools, motion demonstration programs, unit tests, virtual CAN workflow, and verification boundaries. It explains how the project can be exercised without hardware and how bench tests relate to real actuator behavior.
- **Appendix A – ICD Byte-Level Reference** provides the formal byte-level protocol definition for command frames, response frames, command payloads, telemetry frames, status arrays, diagnostic encoding, and validation rules.
- **Appendix B – State Machine Reference** gives the detailed state set, event glossary, transition table, transition diagram, command-to-event mapping, movement gating, stream gating, and safety interpretation of the STC state machine.
- **Appendix C – Configuration Schema, Axis Mapping, and Examples** specifies the INI configuration schema, accepted keys, defaults, validation rules, axis mappings, diagnostic mappings, and canonical configuration excerpts.
- **Appendix D – CANopen and DS402 Mapping** records the Nanotec CANopen and CiA 402 mapping used by the implementation, including PDO identifiers, payload layouts, object-dictionary entries, controlword sequences, statusword interpretation, mode mapping, EMCY handling, heartbeat handling, and commissioning checks.

Taken together, these chapters describe STC as the control core of a distributed turret platform. The goal is to make the full software and hardware-

control stack technically explicit, reproducible, testable, and maintainable.

Taken together, the following chapters describe STC not as an isolated program, but as the control core of a distributed turret platform. The goal is to make the full stack technically explicit, reproducible, and maintainable.

Chapter 2

System Context

2.1 End-to-End System Definition

The Steering Turret Control System is positioned between the external Control Unit and the turret hardware. At system level, the control chain is:

Control Unit → TCP command interface → STC daemon → motion backend → fieldbus / motor protocol network → drives and actuators

The telemetry chain follows the reverse direction:

Drives, actuators, and runtime state → STC daemon → TLMH / TLML UDP telemetry → Control Unit

This placement defines the role of STC in precise terms. The Control Unit is the external command authority and telemetry consumer. The STC daemon is the local execution authority that interprets commands, enforces software rules, supervises state, and interacts with the motion layer. The motion backend is the abstraction boundary that hides the concrete actuator-communication details from the higher runtime. Below that boundary, the system reaches the real devices through CAN-based motor communication, currently centered on the Nanotec CANopen path and extended by the secondary MYACTUATOR library described later in this document.

In the current implementation, command ingress is binary and deterministic over Ethernet/TCP, while telemetry egress is transmitted separately over UDP in two streams: a high-rate stream for fast kinematic observability and a low-rate stream for state, health, and diagnostic context. This separation

keeps the control path simple and synchronous while allowing telemetry to be published at fixed rates without coupling it to command transactions.

2.2 Control Path

From the perspective of system behavior, a command issued by the Control Unit passes through five distinct stages.

1. Control Unit to STC network ingress

The Control Unit sends a framed binary command over TCP/IP to the STC server. The interface is versioned and little-endian, and the default server-side TCP port is configurable, with the default project configuration using port 7000. The command families represented in the current ICD and codec include state-management commands, self-levelling and test commands, turret motion, SUT motion, mirror motion, laser control, storage and reset commands, and information requests.

2. ICD parsing and fast validation

The STC daemon receives the raw TCP frame and decodes it through the ICD codec layer. At this stage the software validates message structure, protocol version, controller identity, command identifier, payload consistency, and immediate state constraints. Commands that are syntactically invalid, disallowed in the current state, or inconsistent with configured limits are rejected before they reach the motion layer.

3. Runtime supervision inside the STC daemon

Once accepted, the command is interpreted in the context of the current software state, configuration, and axis mapping. Purely software-side effects, such as state updates or manual-position data updates, can be applied immediately. Motion-bearing commands are normalized into internal actions that the runtime can execute in a controlled and deterministic way.

4. Motion backend execution

The STC daemon does not directly embed all motor-protocol logic into the command path. Instead, it delegates motion execution to the motion backend abstraction. This backend is responsible for turning normalized turret,

SUT, mirror, stop, or storage requests into concrete device-level actions. The abstraction exists so that the higher runtime remains stable even when the underlying actuator protocol or hardware family changes.

5. Field network and devices

At the hardware side, the backend communicates with the actual drives and actuators over the selected CAN-based low-level path. In the current real-hardware motion path this is a CANopen/DS402 network using SocketCAN on Linux and a Nanotec low-level library. That path supervises per-axis motion, telemetry, heartbeat, and fault signaling. In parallel, the project also contains a complete MYACTUATOR low-level library for the secondary actuator family, which is integrated at the same architectural level even though its protocol model differs from CANopen.

This staged path is important because it makes the software boundary explicit: the Control Unit does not speak directly to drives, and the drive libraries do not own mission-level command semantics. STC is the layer that binds those worlds together.

2.3 Telemetry Outflow

The telemetry path is intentionally split into two streams with different roles.

2.3.1 TLMH: high-rate kinematic telemetry

TLMH is the fast telemetry stream. Its structure is fixed-size and axis-centric. It carries arrays of position, rate, and acceleration values for all configured axes. In the current codec, these arrays are defined for 18 axes, which matches the canonical turret motion model used by the project. The default runtime configuration publishes this stream every 10 ms, corresponding to a nominal 100 Hz rate.

Operationally, TLMH is the stream that gives the Control Unit an immediate view of turret motion. It is the appropriate channel for observing actual axis movement, speed behavior, and dynamic response while commands are being executed.

2.3.2 TLML: low-rate state, health, and diagnostics telemetry

TLML is the contextual telemetry stream. It carries the lower-rate information required to understand the global condition of the system rather than the instantaneous kinematics alone. In the current codec this includes the STC software state, uptime, environmental fields, manual-position or navigation-related data, motor status bytes, laser status bytes, self-levelling related values, and a variable-length diagnostic/error list. The default runtime configuration publishes this stream every 100 ms, corresponding to a nominal 10 Hz rate.

Operationally, TLML is the stream that tells the Control Unit whether the system is healthy, which operating state it is in, what the discrete outputs currently look like, and whether any errors or degraded conditions are active.

2.3.3 Role of the STC daemon in telemetry generation

The STC daemon is responsible for fusing three classes of information into telemetry:

- the latest drive and actuator feedback gathered through the motion layer,
- the current software state and command-execution context,
- the configuration-defined interpretation of axes, devices, and outputs.

This is an important architectural point. Telemetry is a system-level product assembled by the STC runtime so that the Control Unit receives values that are already interpreted in the language of the turret domain.

2.4 Internal Runtime View of the System

From inside the STC software, the end-to-end system is decomposed into four cooperating layers.

2.4.1 Interface layer

The interface layer terminates the external binary protocol. It is responsible for receiving command frames, decoding them, validating them, building ACK/NACK responses, and packing telemetry frames. This layer is protocol-facing, not motor-facing.

2.4.2 Runtime orchestration layer

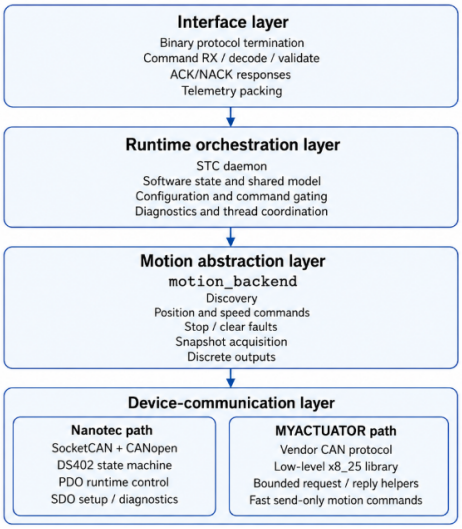
The runtime orchestration layer is the actual STC daemon. It owns the software state, shared model, configuration, command gating, diagnostics, and thread coordination. It is the layer that decides what a command means under the current operating conditions.

2.4.3 Motion abstraction layer

The motion abstraction layer is expressed as the `motion_backend` interface. It defines the operations that the upper runtime expects from the motion subsystem: discovery, position moves, speed commands, stop behavior, fault clearing, snapshot acquisition, and discrete outputs such as lasers. This layer is what allows the system to remain coherent even when the hardware-facing implementation changes.

2.4.4 Device-communication layer

The lowest layer contains the concrete low-level libraries and their bus interactions. On the Nanotec path, this is a SocketCAN-based CANopen implementation with DS402 state-machine behavior, PDO-based runtime control, and SDO-based setup and diagnostics. On the MYACTUATOR path, the project uses the vendor CAN protocol through a dedicated low-level library with bounded request/reply helpers and send-only fast-path motion commands. These protocol-specific details are intentionally isolated from the higher STC semantics.



2.5 Domain Model of the Turret Configuration

The turret software is configured through an axis-centric domain model. The central modeling decision is that motion is represented first as a set of numbered axes, and only then as turret-level or payload-level functional objects.

The basic configuration entities are the following.

2.5.1 Axes

An `AxisN` section is the primitive motion object of the system. Each axis defines:

- the bus/node identity used to reach the device,
- conversion parameters such as counts-per-revolution and gear ratio,
- angular limits in engineering units,
- maximum speed,
- storage position,
- motion-profile parameters,
- diagnostic category and subcategory metadata.

This means that every controllable motion degree of freedom is first represented as a physical or logical axis with its own conversion and safety envelope.

2.5.2 Turret and SUT functional objects

The turret itself is modeled as a two-axis object:

- one azimuth axis,
- one elevation axis.

In the current canonical configuration, the turret azimuth motor controls the mast of the turret, while the turret elevation motor controls the infrared camera mounted on the turret. At protocol level these two degrees of freedom are grouped as a single turret object, but at configuration and motion-execution level they remain two independent axes with separate limits, ratios, and status.

Each SUT is also modeled as a paired two-axis functional object:

- one azimuth axis for the corresponding SUT,
- one elevation axis for the corresponding SUT mirror,
- one discrete laser state associated with that SUT.

This grouping is fundamental because it allows the Control Unit to address `SUT1` through `SUT8` as semantic payload objects instead of reasoning directly about raw axis indices. The software therefore exposes SUT control in functional terms, while the backend still executes the request through the underlying mapped axes.

2.5.3 Software-visible payload groups

A payload group is represented by configured azimuth and elevation axes. The software maps logical IDs to the selected axes and checks state, limits, and availability before motion. Hardware payload-design variants are omitted from this edition.

2.5.4 Mirrors and lasers

Each mirror is modeled as a single-axis elevation object. Mirror commands therefore select one mirror or all mirrors and act on the corresponding mapped elevation axes.

In the canonical turret design, the SUT alignment laser is attached to the corresponding SUT mounting bracket and is controlled by software only as a discrete on/off output. Its practical role is calibration and target indication: the emitted laser is directed toward the mirror so that the reflected spot indicates where the corresponding SUT is targeting. For this reason, mirrors and lasers are distinct software objects but they are physically and operationally related during calibration and alignment procedures.

A crucial point is that mirror identifiers and laser states are part of the same configuration domain model as the motion axes. They are not hidden device details. This is what makes the control semantics inspectable and reproducible.

In the current software implementation, laser control is represented as a commanded software state associated with each SUT, but no dedicated laser hardware backend is integrated in the runtime. Consequently, the `LASER` command updates the internal commanded state, and TLML `laser_status` reflects that commanded state rather than a confirmed hardware feedback path.

2.6 Configuration File as Canonical Source

The axis definitions, functional mappings, and discrete-output associations described in this chapter are defined through the project configuration file:

`STC/dev/config/stc_default.ini`

This file is the canonical configuration reference for the current system model. It declares the 18 motion axes, assigns each axis to a node ID, defines motion limits and scaling parameters, maps the turret, SUT, mirror, and laser objects, and provides diagnostic metadata used by the runtime. The full configuration schema, accepted keys, defaults, validation rules, and canonical examples are given in Appendix C.

The important architectural point is that the meaning of objects such as `Axis0`, `SUT7`, or `Mirror5` is not hidden only inside source code. It is written in a versioned configuration artifact that can be inspected, reviewed, modified,

and validated independently from the command-processing logic.

A short extract of the default configuration is shown below.

```
[Axis0]
node_id = 1
counts_per_rev = 4096
gear_ratio = 100
min_deg = -140
max_deg = 140
max_speed_deg_s = 45
storage_deg = 0
pp_profile_vel = 500

[Axis1]
node_id = 2
counts_per_rev = 4096
gear_ratio = 20
min_deg = -5
max_deg = 180
max_speed_deg_s = 45
storage_deg = 0
pp_profile_vel = 500

[Turret]
az_axis = 0
el_axis = 1

[SUT1]
az_axis = 2
el_axis = 10
laser_on = 0

[Mirror1]
axis = 10
```

Listing 2.1: Extract from the default STC configuration

2.7 Canonical 18-Axis Reference Configuration

The default project configuration provides the canonical reference example for the turret domain model. It defines 18 motion axes, numbered `Axis0` to

`Axis17`, and maps them into turret motion, eight SUTs, eight mirrors, and eight laser outputs.

The mapping is:

- `Axis0`: turret azimuth, i.e. mast rotation
- `Axis1`: turret elevation, i.e. infrared camera elevation
- `Axis2` to `Axis9`: SUT azimuth axes for `SUT1` to `SUT8`
- `Axis10` to `Axis17`: SUT elevation axes for `SUT1` to `SUT8`, also used as the canonical mirror-axis group

The same configuration model also defines:

- `Mirror1` to `Mirror8` mapped to `Axis10` to `Axis17`
- software laser-state fields for `SUT1` to `SUT8`
- per-axis diagnostic category and subcategory metadata for TLML diagnostic packing

The current ICD uses `motor_status[18]` in canonical axis order. This keeps telemetry aligned with the same 18-axis model used by TLMH position, rate, and acceleration arrays. The shipped default file declares the full 18-slot domain model and explicitly enables only the axes intended by that file. Hardware deployment profiles shall enable additional axes only after node IDs, driver families, limits, and diagnostic metadata are verified.

2.7.1 Canonical mapping table

Table 2.1 summarizes the default configuration used as the system reference.

Table 2.1: Canonical default STC axis and functional-object mapping

Functional Object	**Azimuth Axis**	**Elevation Axis**	**Mirror Axis**	**Laser**
Turret	Axis0	Axis1	–	–
SUT1	Axis2	Axis10	Mirror1 → Axis10	Laser1
SUT2	Axis3	Axis11	Mirror2 → Axis11	Laser2
SUT3	Axis4	Axis12	Mirror3 → Axis12	Laser3
SUT4	Axis5	Axis13	Mirror4 → Axis13	Laser4
SUT5	Axis6	Axis14	Mirror5 → Axis14	Laser5
SUT6	Axis7	Axis15	Mirror6 → Axis15	Laser6
SUT7	Axis8	Axis16	Mirror7 → Axis16	Laser7
SUT8	Axis9	Axis17	Mirror8 → Axis17	Laser8

2.7.2 Axis-group parameters

The default configuration is also regular enough that the 18 axes can be described by four parameter groups rather than 18 unrelated entries. This grouped view is more useful for both implementation and review.

Table 2.2: Axis groups and main default configuration values

Axis Group	**Indices**	**Node IDs**	**Gear Ratio**	**Range [deg]**	**Max Speed [deg/s]**
Turret mast azimuth	Axis0	1	100	[-140, 140]	25
Turret IR camera elevation	Axis1	2	20	[-5, 180]	25
SUT azimuth group	Axis2–Axis9	3–10	20	[-35, 35]	25
SUT elevation / mirror group	Axis10–Axis17	11–18	20	[-12.5, 18.5]	25

For all 18 axes in the default configuration, the following values are common:

- `counts_per_rev = 4096`
- `storage_deg = 0`
- `pp_profile_vel = 500`
- `diag_cat = 0x06`
- `diag_subcat = 0x3F`

This grouped regularity is not only convenient for documentation. It also explains why the software can treat the turret as a structured multi-axis mechanism rather than as 18 arbitrary channels.

2.8 Chapter Summary

The system context of STC is therefore defined by two complementary structures. The first is the end-to-end control and telemetry chain linking the Control Unit, the STC daemon, the motion backend, and the hardware network. The second is the configuration-defined domain model in which 18 numbered axes are projected into turret motion, SUT motion, mirror motion, and laser outputs.

The configuration file `dev/config/stc_default.ini` is the concrete bridge between those two structures: it gives the runtime a description of what each axis means, which limits apply, and how the physical turret is presented as a set of controllable software objects.

Chapter 4

Architecture Overview

4.1 Purpose of this Chapter

This chapter gives a compact view of the active STC software architecture. It explains how the repository is organized, which build products exist, and how the main runtime is separated into core control logic, motion abstractions, and supporting tools. The focus is the current `dev/` codebase and the CMake build graph that produces the operational server, protocol tools, optional GUI, simulator path, and validation targets.

4.2 Repository Structure and Build Products

The active software baseline is centered on the following repository areas:

- `dev/include/` contains the public interfaces of the runtime modules, including the ICD API, domain model, configuration schema, state machine, diagnostics, low-level motor libraries, and the motion backend interface.
- `dev/src/` contains the main runtime implementation, including `stc_server`, logging, ICD codec, model and configuration logic, action handling, limits, state logic, faults, diagnostics, and the real CAN backend.
- `dev/src/simulation/` contains the simulator backend used when the software runs without real motion hardware.
- `dev/tools/` contains operator and developer tools such as the TCP command injector, the UDP telemetry receiver, and the optional Qt

diagnostic GUI.

- `dev/tests/` contains unit, integration, and concurrency tests.
- `dev/config/` contains the default runtime configuration used by the server.
- `deploy/` contains deployment assets such as the service unit, preflight checks, and CPU-affinity guidance.

The CMake build produces the products shown in Table 4.1.

Table 4.1: Main build products of the active STC codebase

Target	**Category**	**Purpose**
`stc_server`	Core runtime	Main STC daemon. Owns TCP command handling, shared runtime state, action execution, telemetry publishing, and motion-backend integration.
`icd_tx_tcp`	Utility	Sends valid ICD commands to a running server for development and integration testing.
`tlm_rx_udp`	Utility	Receives and decodes TLMH and TLML telemetry streams.
`diag_gui_qt`	Utility	Optional Qt diagnostic GUI built when Qt6 is available.
`simulator`	Abstraction support	Static library used in simulator builds of the server and selected tests.
`test_actions`, `stc_state_test`, `test_icd_codec`, `stc_config_test`, and related tests	Validation	Focused test targets for actions, state logic, diagnostics, configuration, logging, and concurrency.

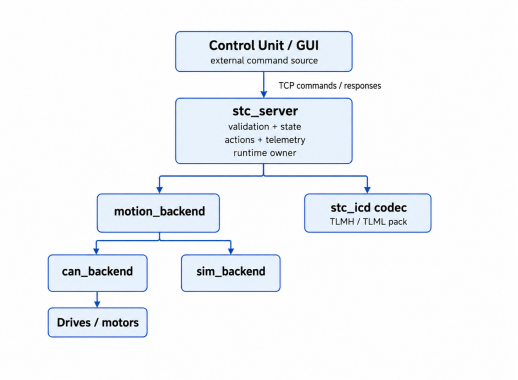
At architectural level, the codebase can be read through three compact groups:

- **Core modules** such as `stc_server`, `stc_icd`, `stc_model`, `stc_config`, `stc_actions`, `stc_limits`, `stc_state`, `stc_faults`, `stc_diag`, and `stc_log`
- **Abstraction modules** such as `motion_backend`, `can_backend`, `sim_backend`, and the low-level motor libraries
- **Utilities** such as the CLI tools, GUI, tests, and deployment scripts

This grouping is sufficient to understand the build graph without overloading the reader with repeated file-by-file discussion.

4.3 Top-Level Runtime Structure

The server runtime can be summarized as a layered flow from external commands to internal actions, motion execution, and telemetry publication.



This is the most important architectural view of the project. The server owns protocol handling, shared state, action flow, and telemetry. The motion backend owns the hardware-facing motion behavior through a stable interface.

4.4 Motion Backend Interface and Build Variants

The motion backend is the clean seam between the server and the concrete motion implementation. The interface in `motion_backend.h` defines:

- a backend configuration structure with CAN settings, watchdog and timeout values, full axis configuration, and node-ID mapping
- a snapshot structure with bus health, per-axis availability, discovered and connected nodes, last-reception timestamps, raw motion values, DS402 status fields, heartbeat state, and EMCY information
- a compact set of operations for startup, stop, discovery, position moves, speed commands, quick stop, fault clearing, laser control, and snapshot retrieval

This contract is implemented through two runtime paths:

- `can_backend.c` for real hardware. It dispatches each enabled axis to the driver selected in configuration, currently `pd4c` for the Nanotec CANopen path and `myact_x8_25` for the MYACTUATOR CAN path.
- `sim_backend.cpp` for simulation. It uses software motor models while preserving the same backend snapshot structure consumed by the server.

The build selects the simulator path or hardware path through `USE_SIMULATOR`. In hardware builds, driver selection remains per-axis. This allows one server process to supervise mixed Nanotec and MYACTUATOR deployments through the same motion-backend contract.

Within the real hardware path, the CAN backend delegates low-level DS402 and SocketCAN behavior to `pd4c`. The detailed behavior of the low-level libraries is documented in their dedicated chapters, so here it is sufficient to note that they occupy the device-facing layer beneath the backend abstraction.

Taken together, the repository structure and build graph show a compact architecture with one operational control daemon, one stable motion contract, two interchangeable motion implementations, and a small set of tools around them for testing, monitoring, and deployment.

Chapter 5

Operating System and Real-Time Environment

5.1 Purpose of this Chapter

This chapter documents the Linux execution environment required by the active STC runtime. The focus is practical deployment on the RockPro64 target and the specific operating-system mechanisms that make the server deterministic and maintainable: thread scheduling policy, startup privilege checks, service deployment, CPU placement, and SocketCAN integration. The goal is to describe the runtime envelope in an operator-actionable way.

5.2 Target Operating-System Baseline

The operational baseline is an embedded Linux installation on the RockPro64 target board. The documented project image uses Manjaro Minimal ARM with the Rockchip RK3399 platform, static Ethernet configuration, and a service-oriented deployment under `/opt/stc`. The runtime assumes:

- a valid SocketCAN interface such as `can0`
- a working TCP and UDP networking stack
- `python3` and `iproute2` for preflight and maintenance
- installed STC binaries, configuration, and deployment assets
- service privileges that allow real-time scheduling and raw CAN access

The RockPro64 is based on the Rockchip RK3399 system-on-chip, a 64-bit ARM big.LITTLE platform with two Cortex-A72 cores and four Cortex-A53 cores. The board also provides LPDDR4 memory, Gigabit Ethernet, USB 3.0, PCIe, eMMC or microSD boot options, and GPIO-oriented expansion interfaces. For STC, the most relevant characteristics are the six-core ARM processing model, the reliable wired Ethernet interface for CU communication, and the Linux-compatible I/O expansion path used for fieldbus adapters such as SocketCAN-compatible CAN interfaces.

From the server point of view, Linux is not only a host operating system. It is part of the control design because thread scheduling, raw CAN sockets, capability handling, and interface bring-up directly affect motion and telemetry determinism.

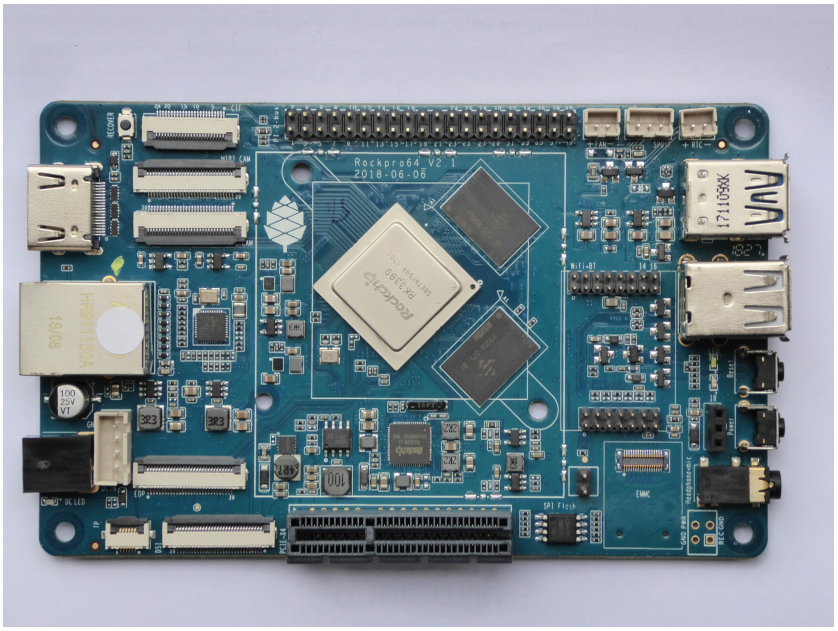


Figure 5.1: ROCKPro64

5.3 Scheduling Model and Real-Time Startup Policy

The STC server supports an explicit runtime scheduling model. Its startup options include:

- `--rt-enabled` to request real-time scheduling

- `--rt-strict` to decide whether missing real-time privileges should abort startup or only degrade scheduling quality
- `--rt-policy fifo|rr|other` to select the scheduling class

Internally, the server uses POSIX thread scheduling attributes and probes the requested policy at startup before the runtime is fully launched. The code explicitly supports `SCHED_FIFO`, `SCHED_RR`, and `SCHED_OTHER`, and it clamps requested priorities through `sched_get_priority_min()` and `sched_get_priority_max()` rather than hard-coding policy limits.

5.3.1 Scheduling-policy meaning

The practical meaning of the supported policies is:

- `SCHED_FIFO` for strict fixed-priority real-time execution. Once a thread is runnable, it keeps the CPU until it blocks, yields, or is preempted by a higher-priority real-time thread.
- `SCHED_RR` for real-time execution with time slicing among equal-priority threads.
- `SCHED_OTHER` for the normal Linux time-sharing class. This is useful for development and fallback operation, though it gives weaker timing guarantees.

For the deployed server, the preferred operational policy is FIFO because the runtime already separates the CAN worker and telemetry publishers into dedicated threads with distinct timing roles.

5.3.2 Priority ordering

The project uses explicit priority ordering between runtime threads. The intended ordering is:

- CAN worker at the highest application real-time priority
- high-rate telemetry thread below the CAN worker
- low-rate telemetry thread below the high-rate telemetry thread

This ordering matches the runtime architecture. The CAN worker owns time-sensitive device interaction and watchdog behavior. The TLMH thread serves the high-rate observability path. The TLML thread serves the lower-rate context and health path.

5.3.3 Failure modes when RT privileges are missing

The server performs an explicit startup probe when real-time mode is requested. If the process lacks the permissions required to switch into the chosen real-time policy, the result depends on the selected startup mode:

- in **strict** mode, startup should fail because the requested runtime envelope is considered mandatory
- in **non-strict** mode, the server may continue with degraded scheduling, typically in the normal Linux scheduling class

The practical failure modes are predictable:

- the process cannot raise thread scheduling policy to FIFO or RR
- requested priority is rejected by the kernel
- telemetry jitter increases
- command-to-execution latency becomes less stable
- watchdog and CAN timing are still logically enforced by software, though the timing margin is reduced

For operations, this distinction is important. A board can still be functional while lacking the intended real-time envelope. It is therefore necessary to distinguish *software correctness* from *deployment correctness*. The preflight and service model described below exist to make that distinction visible before the system is considered ready.

5.4 Service Deployment Model

The preferred deployment model is a managed `systemd` service. The provided unit file launches the server from `/opt/stc`, runs a preflight check before startup, enables automatic restart on failure, and configures the privilege and scheduling envelope required for deterministic execution.

A representative baseline is:

```
[Service]
User=stc
Group=stc
WorkingDirectory=/opt/stc
ExecStartPre=/opt/stc/deploy/stc_preflight.sh
ExecStart=/opt/stc/bin/stc_server --config
   /opt/stc/config/stc_default.ini \
```

```
  --rt-enabled 1 --rt-strict 1 --rt-policy fifo
CapabilityBoundingSet=CAP_SYS_NICE CAP_NET_RAW
AmbientCapabilities=CAP_SYS_NICE CAP_NET_RAW
LimitRTPRIO=90
LimitMEMLOCK=infinity
LimitRTTIME=infinity
CPUSchedulingPolicy=fifo
CPUSchedulingPriority=70
```

Listing 5.1: Key elements of the STC service model

This service model captures four important operating-system requirements.

5.4.1 Capability-based permissions

The service does not need full root privileges for normal operation. The intended privilege model is capability-based:

- `CAP_SYS_NICE` is needed to raise scheduling policy and priority for real-time threads
- `CAP_NET_RAW` is needed to open and bind raw SocketCAN sockets

This is the minimum useful capability set for the deployed runtime. It reduces the privilege surface while keeping the server able to satisfy its real-time and CAN requirements.

5.4.2 Resource limits

The service unit also defines the real-time resource limits expected by the server:

- `LimitRTPRIO=90` to allow high real-time priorities
- `LimitMEMLOCK=infinity` to avoid zero locked-memory allowance
- `LimitRTTIME=infinity` to prevent premature runtime throttling under long-lived real-time execution

These are not optional tuning details. The preflight script checks the same envelope and treats insufficient `RTPRIO` or zero `MEMLOCK` as deployment faults.

5.4.3 Preflight model

Before the service starts, the deployment runs `stc_preflight.sh`. This script performs four checks:

- the requested CAN interface exists
- the interface is a real CAN interface and is already `UP`
- raw SocketCAN access can be opened and bound through Python
- the process has sufficient `RTPRIO` and non-zero `MEMLOCK`

The preflight behavior is important operationally because it converts hidden OS misconfiguration into explicit startup failure. It prevents the service from entering a half-valid state where the binary exists and the configuration file is present, though the runtime environment is still not safe for deterministic control.

5.5 CPU Isolation and Affinity Strategy

The project includes explicit CPU-isolation guidance because scheduling policy alone is not enough to guarantee stable timing on a multicore Linux board. Background kernel work, IRQ handling, and unrelated user-space activity can still interfere with the CAN worker and telemetry threads if CPU placement is left fully uncontrolled.

The recommended baseline is:

- reserve at least one isolated core for the STC process
- pin the service to the isolated core or isolated cores through `systemd CPUAffinity`
- keep CAN-related IRQ handling aligned with the CAN worker core where possible
- preserve priority ordering between CAN worker, TLMH, and TLML threads

A typical kernel baseline for one dedicated core is:

```
isolcpus=2 nohz_full=2 rcu_nocbs=2
```

Listing 5.2: Example CPU-isolation baseline

A corresponding service-level placement is:

```
CPUAffinity=2
```

Listing 5.3: Example service CPU affinity

If two dedicated cores are available, a practical baseline is to reserve one for the CAN worker and one for telemetry publication:

```
CPUAffinity=2 3
```

Listing 5.4: Example dual-core service CPU affinity

5.5.1 Recommended baseline

For the current STC runtime, the recommended baseline is:

- one isolated core minimum
- FIFO scheduling enabled
- CAN worker at highest application RT priority
- TLMH below CAN worker
- TLML below TLMH
- service pinned to the isolated core or isolated cores
- CAN IRQ affinity reviewed and aligned with the CAN worker core

5.5.2 Validation checklist

The deployment should be validated through a short checklist after installation:

1. Confirm that the service is running with the intended scheduling policy and priority.
2. Confirm that the process is pinned to the intended CPU set.
3. Confirm that the CAN interface is up and reachable through raw SocketCAN.
4. Confirm that CAN IRQs are routed to the intended CPU when hardware and driver support make this possible.
5. Confirm that TLMH and TLML timing statistics remain stable during operation.

Useful validation commands are:

```
ps -eLo pid,tid,cls,rtprio,psr,comm | grep stc_server
cat /proc/interrupts
ip -details link show can0
```

```
4 systemctl status stc_server
```

Listing 5.5: Runtime validation commands

This checklist is small on purpose. It is intended to be used by an operator or integrator during bring-up and after platform changes.

5.6 SocketCAN Conceptual Model

The STC motion path uses Linux SocketCAN. In this model, CAN interfaces are exposed as ordinary network devices such as `can0` or `vcan0`. User space interacts with them through the socket API rather than through a custom character driver.

For the STC runtime, this means:

- the CAN bus is configured through standard Linux networking tools
- the server and low-level libraries open CAN sockets through `PF_CAN`
- the PD4-C library uses raw CAN sockets through `SOCK_RAW` and `CAN_RAW`
- bus filtering, read, write, and polling follow the same general model as other Linux socket types

This matches the design of `pd4c`. The library is intentionally single-threaded and owns no internal worker thread. Instead, the application owns threading and may call `pd4c_poll()` from the CAN worker thread. This is a good fit for SocketCAN because:

- the file descriptor can be integrated directly into a poll-based loop
- receive filtering can be configured at the socket level
- the control daemon can keep ownership of concurrency and timing decisions

From a control-design perspective, SocketCAN is therefore not only a transport mechanism. It is the Linux abstraction that lets the STC server treat CAN as part of the standard OS networking model while still maintaining deterministic user-space control over polling, scheduling, and thread placement.

5.7 Why Raw CAN Access Matters

The service capability set includes `CAP_NET_RAW` because the motion path relies on raw CAN sockets. This is required for the low-level behavior implemented by `pd4c` and the CAN backend:

- transmitting RPDO and SDO frames directly
- receiving TPDO, heartbeat, and EMCY frames directly
- using socket-level filtering and polling
- keeping the library lightweight and close to the actual bus behavior

In practice, if raw CAN access is missing, the preflight script will fail during its bind check and the service should never be considered ready. This makes CAN permission issues visible at startup rather than later during motion execution.

5.8 Operational Baseline

A compact operational baseline for the current project is therefore:

- RockPro64 with the documented Manjaro Minimal ARM image
- static network configuration
- configured and up `can0` interface
- deployed `systemd` service with capability-based permissions
- successful preflight check
- FIFO scheduling enabled
- CPU affinity configured for at least one dedicated core
- validated SocketCAN access and runtime timing behavior

This baseline is sufficient for repeatable bring-up, service deployment, and deterministic runtime operation of the active STC server.

Chapter 6

Communication Protocol

6.1 Purpose of this Chapter

This chapter defines the software communication contract between the Control Unit and the Steering Turret Control System. The ICD is the external boundary of STC. It defines which messages are accepted, how they are encoded, how responses are returned, and how telemetry is published.

The main chapter keeps the protocol readable and implementation-oriented. Full byte-level tables, field offsets, command identifiers, response codes, payload sizes, and telemetry layouts are collected in Appendix A.

6.2 Communication Model

The STC communication model uses one command channel and two telemetry channels:

- **TCP command and response channel** for deterministic request and response exchange between the Control Unit and `stc_server`
- **UDP TLMH stream** for high-rate kinematic telemetry
- **UDP TLML stream** for low-rate state, health, and diagnostic telemetry

The command channel is transaction-oriented. Each command frame carries a sequence number and receives one response frame with the same command identity and sequence number. The telemetry channels are periodic and publisher-driven. They do not require request frames from the Control Unit.

The final protocol uses one external controller role: the Control Unit. The `controller_id` field remains in the frame for compatibility and traceability, although the operational contract treats the Control Unit as the authoritative command source.

At runtime, telemetry is sent by UDP unicast to the IP address of the currently connected TCP client. While a Control Unit session is connected, that client IP is the authoritative destination for both TLMH and TLML. This avoids a separate telemetry registration step and keeps bench testing simple.

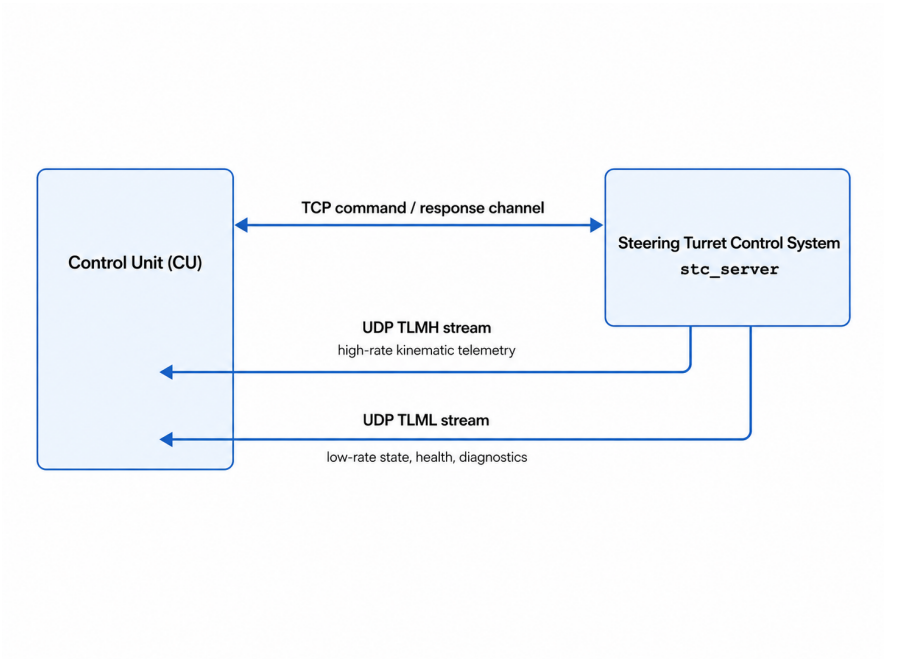


Figure 6.1: Communication Overview

6.3 Protocol Encoding Rules

All ICD frames use deterministic binary encoding. The current protocol version is `0x01`. All fields are encoded in little-endian format. The codec uses explicit read and write helpers for integer and floating-point fields rather than direct structure casting. This keeps the wire format independent from compiler padding and host structure layout.

The common encoding rules are:

- all multi-byte integer fields are little-endian

- `float` fields are IEEE-754 single precision encoded as little-endian 32-bit values
- `double` fields are IEEE-754 double precision encoded as little-endian 64-bit values
- command and response payloads are bounded by `STC_ICD_MAX_PAYLOAD`
- telemetry axis arrays use `STC_ICD_AXES_COUNT = 18`
- TLML diagnostic lists are bounded by `STC_ICD_MAX_ERRORS`

The magic constants are shown in Table 6.1.

Table 6.1: ICD magic constants

Frame Type	**ASCII Magic**	**Meaning**
Command	`TCMD`	TCP command frame from CU to STC
Response	`TRSP`	TCP response frame from STC to CU
Telemetry High	`TLMH`	UDP high-rate telemetry frame
Telemetry Low	`TLML`	UDP low-rate telemetry frame

6.4 TCP Command and Response Contract

The TCP channel carries command frames from the Control Unit to STC and response frames from STC to the Control Unit. TCP is used because command execution requires ordered delivery, complete-frame reconstruction, and deterministic acknowledgement.

6.4.1 Command frame

A command frame contains:

- magic constant `TCMD`
- protocol version
- controller identifier
- command identifier

- sequence number
- sender timestamp in milliseconds
- payload size
- command-specific payload

The current command header size is 20 bytes. The payload follows immediately after the header. The payload size field defines exactly how many payload bytes must be present.

The command identifier selects the payload interpretation. Examples include `CU_STATE`, `SELFLEV`, `IBIT`, `STORAGE`, `LASER`, `TURRET`, `MIRROR`, `SUT`, `RESET`, `CONFIG`, `MANUALPOS`, `POSMODE`, and the request commands for software version, hardware information, and work time.

6.4.2 Response frame

A response frame contains:

- magic constant `TRSP`
- protocol version
- command identifier
- sequence number
- result code
- error code
- payload size
- optional response payload

The current response header size is 14 bytes. The result field is either `ACK` or `NACK`. The error code explains the reason for a rejected command or reports `NoError` for accepted commands. Some commands return an additional payload, for example version, hardware information, work time, and configuration information requests.

6.4.3 Sequencing

The sequence number is generated by the sender and copied into the response. TCP already provides ordered transport, while the sequence number gives the Control Unit a simple way to match responses to commands, detect missing

responses, and support scripted bench tests. Sequence numbers are 16-bit values and wrap naturally after `65535`.

6.5 Payload Semantics

The ICD payloads are command-specific and compact. Table 6.2 summarizes the main payload groups. The byte-level layouts are defined in Appendix A.

Table 6.2: Command payload summary

Command Group	**Payload Meaning**
`CU_STATE`	Requested high-level state or transition command
`SELFLEV`, `IBIT`, `POSMODE`	One-byte mode or start-stop selector
`STORAGE`, `RESET`, `DELLOG`, `FACTORY`, `SELFTEST`, `SHUTDOWN`	Empty payload lifecycle or maintenance command
`LASER`	Laser identifier and on/off selector
`TURRET`	Motion type plus azimuth and elevation values as 32-bit floats
`SUT`	SUT identifier, motion type, azimuth value, and elevation value
`MIRROR`	Mirror identifier, motion type, and elevation value
`CONFIG`	Empty request payload. Response payload returns the active configuration name and path
`MANUALPOS`	Four 64-bit doubles carrying latitude, longitude, altitude, and heading
`REQ_SW_VERSION`, `REQ_HW_INFO`, `REQ_WORK_TIME`	Empty request commands that return response payloads

Motion-bearing commands use the same motion-type enumeration:

- `0` for relative motion
- `1` for absolute motion
- `2` for speed motion

Relative and absolute motion values are expressed in degrees. Speed values are expressed in degrees per second. These values refer to external load movement, not internal motor-shaft rotation.

The server validates motion type, selected object identifier, payload size, state-gate legality, and configured limits before forwarding the command to the action and motion layers. If validation fails and the server can recover the command identifier and sequence number, it returns a structured `NACK` with a specific error code. This best-effort NACK behavior is part of the formal contract because it makes protocol integration observable during malformed-frame testing.

6.6 UDP Telemetry Contract

STC publishes two UDP telemetry streams: `TLMH` and `TLML`. The streams serve different operational purposes and have different rates. Both are sent by UDP unicast to the active TCP client IP.

6.6.1 TLMH high-rate telemetry

`TLMH` is the high-rate kinematic telemetry stream. Its default rate is 100 Hz, corresponding to a nominal period of 10 ms.

The frame contains:

- magic constant `TLMH`
- protocol version
- telemetry sequence number
- timestamp in milliseconds
- position array for 18 axes
- rate array for 18 axes
- acceleration array for 18 axes

This stream is intended for motion observability. It gives the Control Unit the current position, velocity, and acceleration view of the configured turret axes.

6.6.2 TLML low-rate telemetry

`TLML` is the low-rate state, health, and diagnostics telemetry stream. Its default rate is 10 Hz, corresponding to a nominal period of 100 ms.

The frame contains:

- magic constant `TLML`
- protocol version
- controller identifier
- telemetry sequence number
- timestamp in milliseconds
- current STC state
- uptime in seconds
- environmental fields
- latitude, longitude, altitude, and heading
- eighteen motor-status bytes
- eight laser-status bytes
- self-levelling values
- diagnostic error count
- diagnostic error list

This stream is intended for operational context. It reports the software state, health condition, degraded or fault indications, laser state, navigation context, and diagnostic state.

6.7 ICD Tooling

The repository provides two command-line ICD tools: `icd_tx_tcp` for TCP command injection and `tlm_rx_udp` for UDP telemetry reception. Both tools use the same ICD codec as the server, which makes them suitable for protocol validation, bench testing, scripted integration tests, and debugging over SSH.

At communication-contract level, these tools verify that valid commands receive `ACK`, rejected commands receive `NACK` with a meaningful error code,

TLMH reflects accepted motion, and TLML reflects state, health, laser status, and diagnostics.

The detailed operational use of these tools, including examples and their role in verification workflows, is documented in Chapter 9.

6.8 Formal Contract Boundary

The ICD is the formal boundary between the external Control Unit and STC. The Control Unit is responsible for sending valid frames according to this contract. STC is responsible for parsing those frames, validating them against protocol and state rules, producing deterministic responses, and publishing telemetry using the same protocol version and encoding rules.

The full byte-level contract is maintained in Appendix A.

Chapter 7

Server Runtime

7.1 Purpose of this Chapter

This chapter documents the internal runtime behavior of `stc_server`. The server is the software heart of STC because it connects the external ICD, the state machine, the action queue, the motion backend, telemetry publication, safety supervision, and diagnostic reporting into one deterministic runtime.

The focus here is the behavior inside the daemon: thread ownership, shared-state protection, queued action execution, state gating, fail handling, watchdog behavior, degraded-mode support, BIT logic, and observability. The external protocol is defined in Chapter 6, while the low-level motor libraries are described in their dedicated chapters.

7.2 Runtime Thread Model

The server uses a deliberately small thread model. Each thread has a clear ownership boundary and a narrow role.

Table 7.1: Main runtime threads

Thread	**Main Responsibility**	**Owned or Accessed State**
TCP command thread	Accepts the active CU connection, receives command frames, parses and validates ICD payloads, applies fast software-side effects, and enqueues motion or lifecycle actions.	Shared runtime state, ICD codec, action queue
CAN worker thread	Owns the motion backend context, executes queued actions, polls backend snapshots, supervises stream watchdogs, and issues quick-stop actions when required.	Motion backend, action queue, shared telemetry fields
TLMH telemetry thread	Publishes high-rate kinematic telemetry at the configured period.	Read-only copy of shared axis telemetry and timing counters
TLML telemetry thread	Publishes low-rate state, health, laser, navigation, and diagnostic telemetry at the configured period.	Read-only copy of shared state, health, and diagnostic data
Optional diagnostics endpoint	Exposes local read-only runtime diagnostics in JSON form when enabled.	Read-only copy of shared counters and health fields

The key invariant is that only the CAN worker owns and calls the active motion backend context. The TCP path never drives the backend directly. It validates commands and converts them into queued actions. This keeps blocking device I/O away from the command-receive path and gives motion execution a single owner.

7.3 Shared-State Design

The server has one central shared runtime structure protected by a mutex. This structure contains the information that must be visible across command handling, action execution, telemetry publication, diagnostics, and safety supervision.

The shared state contains:

- per-axis configuration, node IDs, storage positions, diagnostic categories, and diagnostic subcategories
- per-axis telemetry such as raw position, raw velocity, statusword, mode display, position in degrees, velocity in degrees per second, and acceleration
- per-axis stream state with enable flag, desired speed, and last command time
- saturation flags, axis availability flags, expected-axis flags, enabled flags, and fault flags
- configured turret, SUT, mirror, and laser mappings
- current STC state and manual-position context
- heartbeat, EMCY, TPDO timestamps, and CAN health indicators
- active CU address and connection state used for telemetry destination selection
- TLMH and TLML sequence counters
- command-path timing and telemetry timing counters
- pending IBIT and self-levelling completion timers
- fail-stop pending state and the fail-stop axis mask

The mutex protects the consistency of this shared runtime model. The telemetry threads lock the state only long enough to copy the values needed for one outgoing packet. The command path locks the state for validation and fast updates. The CAN worker locks the state when it updates backend snapshots, stream state, fault state, and telemetry fields.

The runtime also uses a small number of atomic variables for process-level shutdown and socket lifetime coordination. These variables include the stop flag and the listener or active-client file descriptors. They are atomic because they are touched by signal handling and by the main runtime path.

The practical invariant is simple: large domain state is mutex-protected, shutdown state is atomic, and backend I/O is owned by the CAN worker.

7.4 Action Queue

The action queue is a first-order architectural component. It is the boundary between command acceptance and motion execution.

The queue stores normalized `stc_action_t` objects. Each action contains:

- an action type
- an axis mask identifying the affected axes
- optional target positions or speed values
- optional state or laser data
- timing fields used for command-path observability

The queue capacity is fixed at `STC_ACTIONQ_CAPACITY = 1000`. This avoids dynamic growth during runtime and makes overload behavior explicit. If the queue is full, push fails and the command path can return an error instead of silently accumulating unbounded work.

The action queue is protected by its own mutex and condition variable. It also uses an `eventfd` wakeup descriptor. This dual mechanism gives the implementation two useful properties:

- condition-variable wakeup is simple for ordinary worker synchronization
- `eventfd` can be integrated into poll-based or file-descriptor-driven worker loops

7.4.1 Action types

The action model includes the following action types:

- `ACTION_SET_STATE`
- `ACTION_PP_MOVE`
- `ACTION_STREAM_SPEED`
- `ACTION_STOP`
- `ACTION_QUICK_STOP`
- `ACTION_ENTER_STORE`
- `ACTION_RESET`

- ACTION_FACTORY
- ACTION_SHUTDOWN
- ACTION_CAN_REDISCOVER
- ACTION_CLEAR_ERRORS
- ACTION_SET_LASER

This keeps the server command path independent from backend-specific function calls. The TCP thread creates actions in STC language. The CAN worker executes them in backend language.

7.4.2 Keep-latest stream policy

Streamed speed commands are handled with a keep-latest policy. When a new ACTION_STREAM_SPEED is pushed, older queued stream-speed actions are filtered out before the new one is inserted.

This policy is important for real-time control. Speed streaming represents the latest desired velocity, not a historical sequence that must be replayed. Replaying stale speed commands would increase latency and could command motion that the Control Unit no longer wants. By keeping only the newest stream action in the queue, the server preserves the intended control semantics.

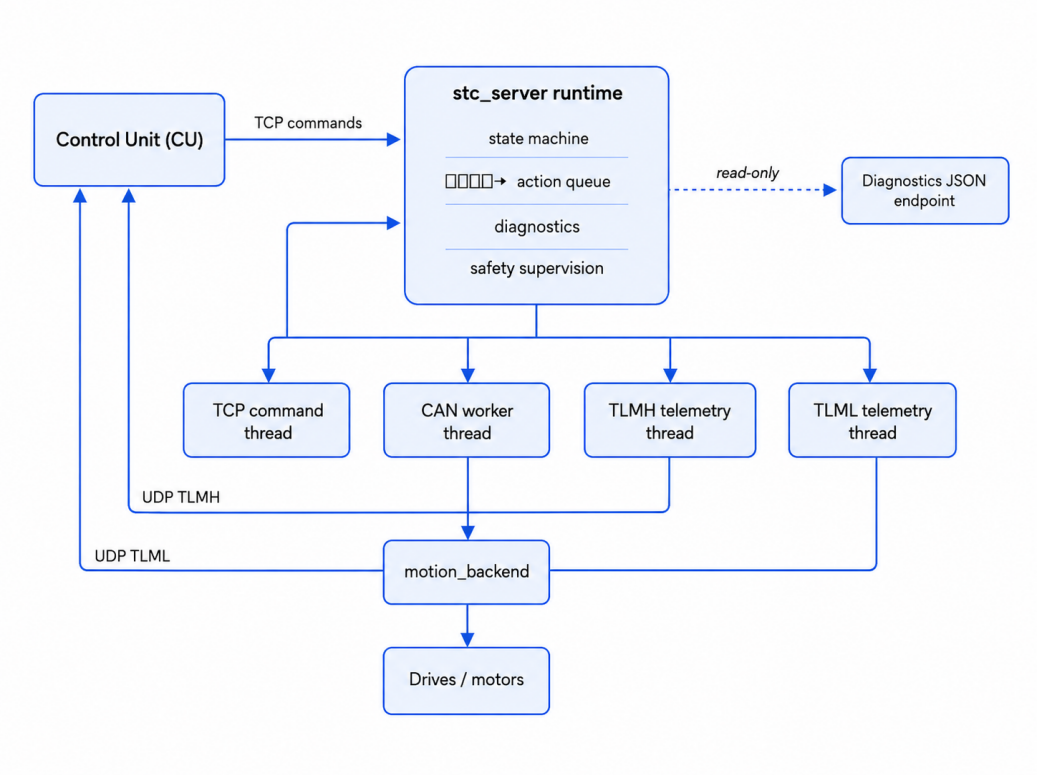
7.5 Command Processing Path

A command received from the CU follows the runtime path below.

1. The TCP thread reads the complete ICD command frame.
2. The ICD codec parses the frame and decodes the known payload.
3. Fast validation checks magic, version, controller role, command ID, payload size, payload values, and immediate state legality.
4. Fast software-side commands update shared state directly when safe to do so.
5. Motion-bearing or backend-bearing commands are converted into actions.
6. The action is pushed to the action queue with timing metadata.

7. The CAN worker wakes, consumes the action, and executes it through the motion backend.
8. The response is returned to the CU as `ACK` or `NACK`.
9. Telemetry threads publish the resulting state, motion feedback, health state, and diagnostic context in TLMH and TLML.

This path keeps the network-facing part responsive. Expensive or blocking work is moved to the worker side. The command response reports acceptance or rejection of the command into the controlled execution path, while telemetry reports the physical and runtime effects.



7.6 State Gating

The runtime state machine is used as an execution gate. The server does not accept every syntactically valid command as executable. The current state controls whether the command can modify motion, streaming, maintenance state, or manual-position context.

The main state rules are:

- motion commands are executable only in `OPER`
- speed streaming is allowed only in `OPER`
- `MANUALPOS` and `POSMODE` are accepted only in `SAFE` or `OPER`
- `SELFLEV` and `IBIT` enter their dedicated running states only through valid state transitions
- `STORE` is entered only through the state transition model
- critical faults force transition to `FAIL`

The state module exposes two explicit predicates: `stc_state_can_move()` and `stc_state_can_stream()`. In the current model both return true only for `OPER`. This makes the motion policy direct and auditable.

When the state changes to one that cannot stream, the runtime clears all per-axis stream enable flags. This prevents a stale speed command from persisting after the system leaves the operational movement state.

The full transition semantics are documented in Appendix B.

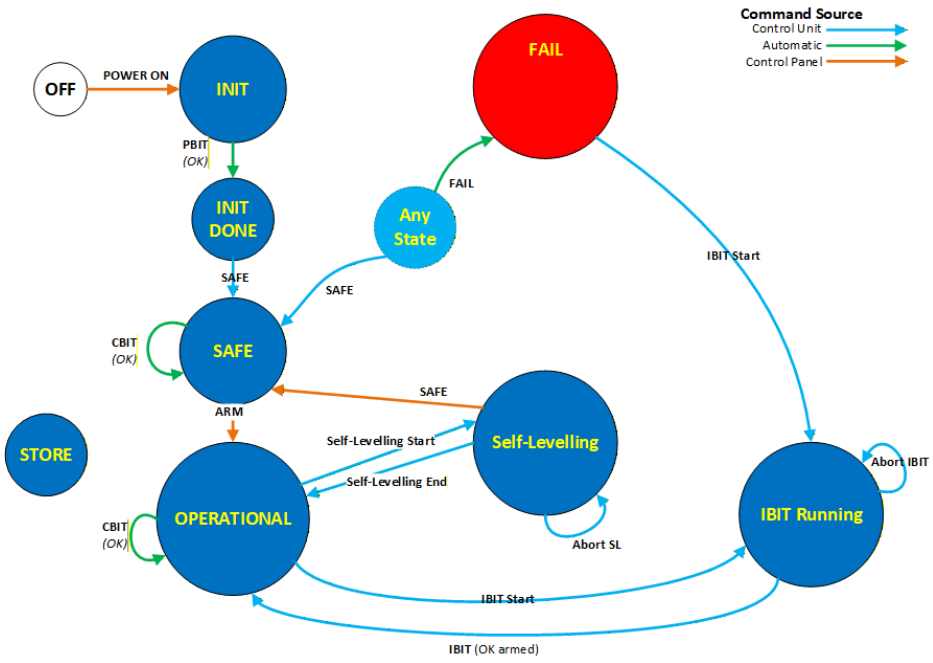


Figure 7.1: State Machine Graph.

7.7 Fault Detection and Severity

The server observes several independent fault and health signals through the motion backend and the shared runtime model. These signals are used for reporting, degraded operation, or transition into `FAIL` depending on their severity.

Table 7.2: Runtime fault and health sources

Source	Runtime Evidence	Interpretation
DS402 statusword	`6041h` statusword bit `0x0008`	The Nanotec drive reports a DS402 fault state. This is an axis fault and can become critical for required axes.
Stale telemetry	`last_rx_ms` compared with `can_timeout_ms`	The axis has no fresh feedback inside the configured timeout. This marks stale communication and supports degraded or fault decisions.
CAN bus health	`can_ok`, discovered nodes, connected-device count	The backend reports whether the fieldbus path is usable and which expected nodes are present.
EMCY frame	`have_emcy`, `emcy_code`, `emcy_reg`	The drive emitted a CANopen emergency message. This is treated as critical evidence for the affected axis.
Heartbeat state	`have_hb`, `hb_state`, `ts_hb_ms`	Heartbeat provides node-liveness context. It is reported and monitored, while heartbeat absence alone is not fatal in the default policy.
Axis saturation	Per-axis saturation flag	A command exceeded configured limits and was clamped. This is informational and must be visible to the external system.
BIT result	PBIT, IBIT, CBIT result values	Test failures drive state-machine events and may force `FAIL`.

Table 7.3: Severity interpretation used by the runtime

Class	Examples	Runtime Response
Informational	Saturated command, heartbeat observation, inactive laser state, non-critical warning	Report through telemetry or log. Continue operation inside configured limits.
Degraded	Loss of azimuth or elevation movement for one or more SUTs, GNSS subsystem failure, partial SUT or mirror capability loss	Continue the safe remaining capability, identify the affected component, report degraded condition to the Control Unit, apply the degraded-mode policy.
Critical	DS402 fault on a required axis, EMCY event, required bus failure, failed BIT, self-levelling timeout, critical CBIT result	Transition to `FAIL`, inhibit ordinary motion, schedule quick stop for relevant axes, publish fault state.

The fault helpers remain intentionally small. DS402 fault detection checks the standard fault bit in the statusword. CAN stale detection compares monotonic time against the last receive timestamp. The higher runtime then decides whether the condition is informational, degraded, or critical according to axis role and system mode.

7.7.1 Degraded operation

Degraded behavior is described here at the software level. Axis availability, feedback age, drive faults, and configured mappings provide the evidence for command eligibility and diagnostics. Hardware-specific minimum configurations and external tracking policies are omitted.

The runtime can report unavailable axes and reject motion on disconnected or faulted axes. Critical faults schedule a stop through the motion worker. Validate the full fault response separately on the intended hardware.

The worker owns backend transactions. Fault handling uses shared snapshots and explicit stop actions to keep ordinary motion and fault-response ordering clear. Physical stopping distance, external interlocks, and complete subsystem qualification require hardware validation.

7.8 FAIL Handling and Safe Stop

The `FAIL` state has priority over ordinary command execution. A critical fault event transitions the software to `FAIL` and marks a fail-stop operation as pending.

During the transition into `FAIL`, the server builds a mask of all axes that are relevant for immediate stopping. The mask includes axes that are expected, available, enabled, or actively streaming. This avoids relying only on one category of state and increases the probability that every moving or potentially enabled axis is included.

The CAN worker consumes the pending fail-stop condition and issues quick-stop behavior through the motion backend. In the Nanotec CAN backend this maps to the low-level DS402 quick-stop path. In the MYACTUATOR path, the equivalent backend behavior is a vendor stop or shutdown command depending on the required stop level and actuator mode. The backend clears enabled state for stopped axes after the stop operation.

This design keeps failure response centralized. The state machine decides that the system is failed. The shared state records which axes require stopping. The CAN worker performs the hardware-facing stop through the backend it owns.

7.9 CAN Worker and Backend Ownership

The CAN worker is the only thread that owns the motion backend. This rule is one of the most important concurrency invariants in the server.

The worker is responsible for:

- starting and stopping the backend
- executing profile-position moves
- executing speed commands
- sending quick-stop commands
- clearing faults
- setting laser outputs through the backend contract
- polling backend snapshots
- updating per-axis availability, telemetry, heartbeat, EMCY, and fault state
- enforcing stream watchdog timeout behavior

It also produces a backend snapshot containing bus health, discovered nodes, connected devices, raw position and velocity, statusword, mode display, heartbeat state, and EMCY state. The server copies backend snapshot data into shared telemetry fields. The telemetry threads then publish this interpreted data through the ICD.

7.10 Telemetry Update Path

The server telemetry path is a readout of the shared runtime model. The CAN worker polls the motion backend, copies the latest backend snapshot into shared axis telemetry, and updates health fields such as availability, motor status, heartbeat state, EMCY state, last-reception time, saturation state, and fault flags.

Telemetry threads do not call the backend directly. They take short locked copies of the shared state and pack the corresponding ICD frames. This keeps telemetry publication independent from CAN I/O while preserving a consistent view of the latest known motion and health state.

The high-rate stream uses the per-axis position, velocity, and acceleration fields. The low-rate stream uses the software state, uptime, navigation context, motor health bytes, laser state, self-levelling fields, degraded or fault context, and diagnostic list. The detailed byte-level telemetry layout is defined in Appendix A.

7.11 Stream Watchdog

Speed commands are stream-oriented. A speed command represents the latest desired velocity for an axis and must be refreshed by the Control Unit while motion is intended.

For every streaming axis, the runtime stores:

- the active stream flag
- the desired speed in degrees per second
- the timestamp of the last valid speed command

The CAN worker supervises these timestamps with monotonic time. If elapsed time exceeds `speed_watchdog_ms`, the worker disables streaming for the affected axis and requests a zero-speed or stop-equivalent command through the motion backend. This converts a lost command stream into a deterministic safe stop.

Active speed streams are also re-issued at a bounded cadence configured by `speed_stream_period_ms`. The refresh cadence keeps valid streams active while avoiding a busy loop. The watchdog timeout and refresh period are therefore complementary parameters: one defines when a stale stream expires, and the other defines how often the backend receives the current commanded speed.

The watchdog is tied to state gating. When the system leaves `OPER`, the runtime clears active stream flags. This prevents an old velocity command from crossing into `SAFE`, `STORE`, `IBIT_RUNNING`, `SELF_LEVELLING`, or `FAIL`.

7.11.1 PBIT

PBIT is the startup test path. It is evaluated during initialization and drives the transition out of `INIT`.

- PBIT pass generates `EVT_PBIT_OK` and allows transition toward `INIT_DONE`
- PBIT failure generates `EVT_PBIT_FAIL` and moves the system to `FAIL`

The current runtime uses configuration validity, backend startup result, expected-axis inventory, CAN health, axis availability, fault state, and identity policy as the practical PBIT evidence. The field `startup_feedback_grace_ms` gives the backend and telemetry path a startup grace window before stale-feedback conditions become fault-eligible.

The policy field `require_identity_for_pbit` controls whether identity evidence is required when evaluating expected hardware. The policy field `allow_degraded_startup` allows a deployment profile to continue into a degraded startup path when the remaining healthy inventory satisfies the selected operating policy.

The final PBIT supervisor should include configuration validation, CAN interface availability, expected-node discovery, drive identity checks, safe initial state checks, and required software-resource checks.

- PBIT pass generates `EVT_PBIT_OK` and allows transition toward `INIT_DONE`
- PBIT failure generates `EVT_PBIT_FAIL` and moves the system to `FAIL`

The final PBIT supervisor should include configuration validation, CAN interface availability, expected-node discovery, drive identity checks, safe initial state checks, and required software-resource checks.

7.11.2 IBIT

IBIT is the requested Built In Test. It is started by command and represented by the `IBIT_RUNNING` state. The server records IBIT as pending and assigns a completion deadline.

The implemented decision basis is the essential runtime health evidence:

- at least one expected axis exists
- CAN is healthy
- all expected axes are available
- no expected axis is faulted

A successful IBIT produces `EVT_IBIT_COMPLETE_OK`. A failed IBIT produces `EVT_IBIT_COMPLETE_FAIL`. Recovery from `FAIL` goes through IBIT, which makes diagnostic recovery explicit.

7.11.3 CBIT

CBIT is the cyclic Built In Test layer. It evaluates runtime health while the server is running. In the current diagnostic module, CBIT can return healthy or critical-fault status for a given shared runtime instance. In the final system, CBIT should aggregate:

- DS402 fault state

- CAN stale status
- EMCY presence
- required-node availability
- temperature and current monitoring
- input-voltage monitoring
- Control Unit and field-interface health
- GNSS or orientation validity

When CBIT reports a critical fault, the runtime should generate `EVT_FAULT_CRITICAL`. The state machine then moves to `FAIL`, and the server schedules quick stop through the normal fail-stop mechanism.

7.12 Runtime Observability

The server includes explicit timing instrumentation because timing behavior is part of the runtime contract.

7.12.1 Telemetry timing

Each telemetry thread records:

- total sample count
- deadline miss count
- expected wake time
- actual wake time
- send completion time
- per-second jitter minimum
- per-second jitter maximum
- per-second jitter average

The implementation uses absolute monotonic sleeps for periodic telemetry scheduling. Jitter is computed as the absolute difference between expected and actual wake time. A wake after the expected time increments the deadline-miss counter.

This gives the operator and developer a direct way to validate whether the board, scheduling policy, and CPU-affinity settings are delivering stable runtime timing.

7.12.2 Command-path timing

The command path records three timestamps:

- TCP receive time
- action queued time
- action execution start time

From these values, the server maintains moving average estimates for:

- TCP receive to queue latency
- queue to execution latency
- end-to-end command acceptance to execution latency

These counters turn command latency into a measurable runtime property rather than an assumption.

7.12.3 Diagnostics JSON endpoint

The optional local diagnostics JSON endpoint belongs to the same observability model. Its role is to expose a read-only view of runtime health for local debugging and integration support.

The endpoint should report only already-owned runtime data, such as:

- current STC state
- active CU connection state
- CAN health and connected-device count
- per-axis availability and last-reception age
- DS402 fault, stale-feedback, EMCY, and heartbeat context
- degraded condition and degraded cause summary
- TLMH and TLML jitter statistics
- command-path latency estimates
- action queue depth and peak depth

- active diagnostic count

The endpoint must not mutate control state, send motion commands, clear faults, or bypass the ICD. It is an observability surface, not a control surface.

7.13 Runtime Invariants

The server runtime is designed around the following invariants:

- only the CAN worker owns the motion backend context
- the TCP command path validates and enqueues motion actions
- shared runtime state is protected by one main mutex
- process shutdown and socket lifetime flags are atomic
- the action queue is bounded
- stream-speed commands use a keep-latest queue policy
- motion and streaming are accepted only in `OPER`
- leaving `OPER` clears streaming state
- entering `FAIL` schedules quick stop across relevant axes
- degraded operation preserves safe remaining capability when the minimum degraded configuration is still available
- telemetry threads read copied shared state and do not call the backend
- runtime timing is measured continuously

These invariants are the practical concurrency and safety contract of `stc_server`. They make the runtime understandable, bounded, and suitable for deterministic control on the RockPro64 target.

Chapter 8

Motion Control over CANopen, DS402, and MYACTUATOR CAN

8.1 Purpose and Motor-Family Allocation

This chapter documents the motion-control layer used by the STC runtime. It explains the standards and protocols used by the motor families, how axis-level commands in degrees and degrees per second are converted into device commands, and how commissioning and calibration should be performed in a controlled way.

The STC motion stack uses two motor-control paths. The Nanotec path is based on CANopen and CiA 402, also known as DS402. The MYACTUATOR path is based on the vendor CAN protocol used by the X8 and L-series actuator families.

The hardware allocation used by the final turret design is:

- Nanotec PD6-C for the main turret azimuth mast motor
- MYACTUATOR V4-X8-120 for SUT motion axes
- MYACTUATOR L-9025 for mirror motion axes

The laboratory development path used Nanotec PD4-C and MYACTUATOR X8-25 units. This remains valid for software bring-up because the PD4-C and PD6-C share the same CANopen and DS402 control model, while the X8-25, X8-120, and L-9025 belong to the same MYACTUATOR CAN protocol

family.

In code, the current MYACTUATOR axis driver name is `myact_x8_25`. This name is a software driver-family label for the implemented MYACTUATOR CAN command set. Deployment documentation may reference X8-25, X8-120, and L-9025 as hardware models, while the configuration key remains the runtime driver selector understood by the parser.

The motor choice is also physical. The main azimuth axis moves the turret mast and benefits from the Nanotec drive model, external gearing, standard DS402 supervision, heartbeat, EMCY, and object-dictionary access. The repeated SUT and mirror axes benefit from compact actuator modules with integrated motor, gearbox, encoder, controller, small physical envelope, sufficient torque for local payload movement, and simple one-frame CAN commands. This reduces wiring, commissioning complexity, and per-axis control overhead for the repeated axes.



Figure 8.1: Nanotec/MYACTUATOR Motors

8.2 CANopen and CiA 402 Primer

CANopen is the higher-level communication profile used by the Nanotec drive path. It defines object-dictionary access, network-management commands, heartbeat monitoring, emergency messages, and process data exchange.

CiA 402 defines the motion-drive behavior on top of CANopen. Its central concept is a finite state automaton for drive power and motion readiness. The application writes the `6040h Controlword` to request transitions, and reads the `6041h Statusword` to observe the current state.

The practical DS402 states used by the library are:

- `Switch On Disabled`
- `Ready To Switch On`
- `Switched On`
- `Operation Enabled`
- `Quick Stop Active`
- `Fault Reaction Active`
- `Fault`

The STC runtime relies on this model for enable operation, quick stop, and fault reset. The library sends the required controlword sequence and waits for matching statusword feedback through TPDO1.

8.3 PDOs, SDOs, and Real-Time Behavior

CANopen provides two main data-exchange mechanisms used by the Nanotec path.

- **PDOs** carry short pre-mapped runtime data. They are used for time-sensitive commands and cyclic feedback.
- **SDOs** access object-dictionary entries by index and subindex. They are used for setup, commissioning, identity reads, diagnostics, and parameter verification.

A PDO can carry up to 8 data bytes in one CAN frame. It can also pack multiple mapped object values into the same frame, such as controlword and mode command, or statusword and mode display. This is why PDOs carry the main real-time traffic in CANopen systems.

The same information can often be reached through SDO access, although the transaction is heavier. An SDO exchange includes command and object-address overhead, and a normal read requires at least one request frame and one response frame. Reading two independent object values through SDO requires two request-response exchanges. With PDO mapping, the same values may arrive in one cyclic or event-driven CAN frame.

For the STC motion path, this distinction is critical. PDOs are used for runtime motion because they have fixed layout, fixed size, low bus overhead, and stable decode cost. SDOs remain in the setup and diagnostic path where additional latency is acceptable.

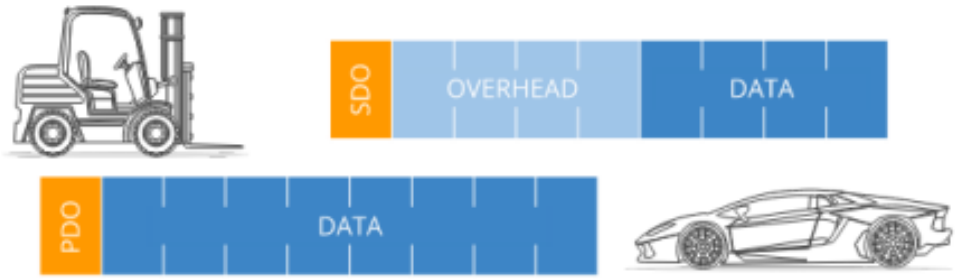


Figure 8.2: PDOs vs SDOs

8.4 Nanotec SocketCAN and PDO Layout

The Nanotec library uses Linux SocketCAN. Each drive node is opened through a raw CAN socket on an interface such as `can0`. The library computes CAN identifiers from the node ID and installs receive filters for the frames needed by that node.

The fixed CANopen identifiers are shown in Table 8.1.

Table 8.1: Nanotec PD4-C and PD6-C PDO and service identifiers

Frame	**CAN ID**	**Mapped Content**
SDO TX	`0x600 + node`	SDO request from STC to drive
SDO RX	`0x580 + node`	SDO response from drive to STC
RPDO1	`0x200 + node`	`6040h Controlword` and `6060h Modes of Operation`
RPDO2	`0x300 + node`	`607Ah Target Position` and `6081h Profile Velocity`
RPDO3	`0x400 + node`	`6042h Target Velocity`
TPDO1	`0x180 + node`	`6041h Statusword` and `6061h Mode Display`
TPDO2	`0x280 + node`	`6064h Position Actual Value`
TPDO3	`0x380 + node`	`6044h Velocity Actual Value`
Heartbeat	`0x700 + node`	NMT heartbeat state
EMCY	`0x080 + node`	Emergency message

This layout is a software invariant. The STC Nanotec path expects the drive PDO mapping to match Table 8.1. A different drive mapping must be treated as a commissioning or configuration issue before operational use.

8.5 Nanotec `pd4c` Library API and Invariants

The `pd4c` library is the low-level Nanotec CANopen library. Although it was developed with a PD4-C device, the API and wire model apply to the PD6-C path used for the turret azimuth mast because both follow the same CANopen and DS402 approach.

The main API groups are:

- **lifecycle**: `pd4c_open()`, `pd4c_close()`, `pd4c_set_logger()`, `pd4c_set_sdo_timeout()`
- **polling**: `pd4c_poll()`, `pd4c_telem()`
- **NMT**: `pd4c_nmt_start()`, `pd4c_nmt_stop()`, `pd4c_nmt_preop()`, `pd4c_nmt_reset_node()`
- **SDO**: `pd4c_sdo_read_u8()`, `pd4c_sdo_read_u16()`, `pd4c_sdo_read_u32()`, `pd4c_sdo_read_i32()`, and the matching write functions
- **PDO**: `pd4c_rpdo1_send_control_mode()`, `pd4c_rpdo2_send_pp()`, `pd4c_rpdo3_send_target_velocity()`

- **DS402**: `pd4c_ds402_enable_operation()`, `pd4c_pp_move()`, `pd4c_ds402_quick_stop()`, `pd4c_ds402_fault_reset()`

The library invariants are:

- the library owns no internal thread
- the application owns threading and calls `pd4c_poll()`
- one context represents one node on one SocketCAN interface
- a context must be protected externally if multiple threads access it
- PDOs are used for runtime motion and feedback
- SDOs are used for setup, commissioning, identity checks, and diagnostics

The STC server respects these invariants by letting the CAN worker own the backend and the backend own the low-level contexts.

8.6 MYACTUATOR CAN Protocol

The MYACTUATOR path is a separate vendor protocol. It does not use CANopen or DS402. It uses compact 8-byte CAN frames with command-specific payloads and fixed identifier bases derived from the node ID.

The low-level library uses these identifier families:

Table 8.2: MYACTUATOR CAN identifier families

Frame Family	**CAN ID**	**Purpose**
Single command	`0x140 + node`	Standard commands such as status, speed, position, PID, acceleration, stop, and shutdown
Single reply	`0x240 + node`	Reply to standard commands
CAN-ID service	`0x300`	Dedicated CAN-ID read and write service
Motion command	`0x400 + node`	MIT-style packed motion command
Motion reply	`0x500 + node`	Packed motion feedback reply

The MYACTUATOR library is designed with the following invariants:

- single-threaded use

- no heap allocation
- no internal worker thread
- transport-agnostic CAN callbacks for send, receive, and monotonic time
- send-only fast-path functions for torque, speed, position, and motion mode
- bounded request and reply helpers for setup, identity, diagnostics, and configuration

This design makes the library portable. It can be used with SocketCAN on the RockPro64, with a PEAK CAN adapter during bench tests, or with another CAN transport layer as long as the callback contract is satisfied.

8.7 MYACTUATOR Runtime Commands

The MYACTUATOR library supports the runtime commands shown in Table 8.3.

Table 8.3: Main MYACTUATOR runtime commands

API Function	Cmd	Purpose
`myact_stop_send()`	`0x81`	Stop motion
`myact_shutdown_send()`	`0x80`	Shut down motor output
`myact_speed_send()`	`0xA2`	Send speed command in `0.01 deg/s` units
`myact_position_abs_send()`	`0xA4`	Send absolute position command with maximum speed
`myact_position_inc_send()`	`0xA8`	Send incremental position command with maximum speed
`myact_motion_send()`	`0x400 + node`	Send packed position, velocity, proportional gain, derivative gain, and feedforward torque

The synchronous variants execute the same command and wait for the expected reply within the configured timeout. The send-only variants are appropriate for fast loops where the backend handles polling and feedback separately.

For the MYACTUATOR path, absolute and relative position commands are not treated as fire-and-forget writes only. The backend computes a physically valid target within axis limits, transmits the corresponding command,

and then polls actuator feedback until the commanded target is reached within a configurable tolerance or a timeout expires. The current runtime therefore depends on `position_tolerance_deg`, `position_timeout_ms`, and `position_poll_ms` as operational parameters of the MYACT position-control path.

8.8 Axis-Level Backend Mapping

The server and ICD express motion in engineering units:

- angles in degrees
- speeds in degrees per second
- axis selection through configured axis indices

The motion backend converts these values into drive-specific units. The conversion is axis-based, using the configuration fields `counts_per_rev`, `gear_ratio`, `vel_scale`, `min_deg`, `max_deg`, and `max_speed_deg_s`.

For the Nanotec path, position conversion follows:

$$\theta_{axis,deg} = \frac{counts \cdot 360}{counts_per_rev \cdot gear_ratio}$$

$$counts = round\left(\frac{\theta_{axis,deg} \cdot counts_per_rev \cdot gear_ratio}{360}\right)$$

Velocity conversion follows:

$$v_{axis,deg/s} = \frac{vel_raw \cdot vel_scale}{gear_ratio}$$

$$vel_raw = round\left(\frac{v_{axis,deg/s} \cdot gear_ratio}{vel_scale}\right)$$

The backend applies profile-position commands by converting the requested target from degrees to counts, enabling DS402 profile-position mode, and sending the target through RPDO2. It applies speed commands by converting degrees per second into target-velocity units, enabling velocity mode, and sending the value through RPDO3.

For the MYACTUATOR path, the standard position and speed commands use vendor engineering units:

- angle commands use `0.01 deg` units
- speed commands use `0.01 deg/s` units
- status feedback includes temperature, current, speed, angle, voltage, and error flags
- motion mode packs position in radians, velocity in radians per second, gains, and torque into one 8-byte command

When a MYACTUATOR command directly represents output-shaft motion, the backend can map STC degrees directly into the vendor units after limit checks. When an external mechanical linkage or extra reduction exists between the actuator and the controlled object, the same axis-level `gear_ratio` concept is used to preserve the STC contract.

8.9 Clamping, Limits, and Saturation

Motion commands are limited before they reach the device layer. The limit logic uses each configured axis envelope:

- minimum angle
- maximum angle
- maximum speed

The default runtime limits declared in `dev/config/stc_default.ini` are shown in Table 8.4.

Table 8.4: Default STC axis limits and motion parameters

Axis Group	**Axes**	**Range [deg]**	**Max Speed [deg/s]**	**Gear Ratio**
Turret azimuth mast	`Axis0`	`-140 to 140`	25	100
Turret infrared camera elevation	`Axis1`	`-5 to 180`	25	20
SUT azimuth axes	`Axis2--Axis9`	`-35 to 35`	25	20
SUT elevation and mirror axes	`Axis10--Axis17`	`-12.5 to 18.5`	25	20

All axes in the default configuration use:

- `counts_per_rev = 4096`
- `storage_deg = 0`
- `pp_profile_vel = 500`

The motion-limit layer clamps commanded target angles to the configured travel envelope of each axis and clamps speed commands to the interval $[-max_speed, +max_speed]$. When a command is clamped, the runtime sets the corresponding axis saturation flag. This allows the server to execute the closest safe command while still reporting that the original request exceeded the configured motion envelope.

For relative motion, the runtime first converts the requested delta into an absolute target using the latest known axis position. It then clamps that absolute target to the configured axis limits and derives the effective relative delta from the clamped result. As a result, the actuator receives only a relative movement that remains compatible with the configured travel bounds.

The low-level libraries also have internal range protection. For example, the MYACTUATOR packed motion command rejects values outside its configured ranges for position, velocity, gains, and feed-forward torque. The preferred control flow is:

1. validate command syntax and state
2. map functional object to axes
3. clamp using STC configuration limits
4. convert engineering units into device units
5. send the device command
6. publish saturation or fault status through telemetry and diagnostics

8.10 Drive Internal Scaling and Mechanical Ratio

The control model separates two concepts that are easy to confuse during bench testing.

- **Drive internal scaling** describes the unit used by the drive feedback or command object. For Nanotec this is the effective count resolution

seen in objects such as `6064h`. For MYACTUATOR this may be `0.01 deg`, encoder counts, or packed radians depending on the command family.

- **Mechanical extra ratio** describes the physical ratio between the motor or actuator output and the controlled turret object.

In the STC axis model, `counts_per_rev` represents the effective drive-side position unit resolution, and `gear_ratio` represents the mechanical ratio from that drive-side rotation to the controlled axis. This distinction is critical for calibration.

A common mismatch occurs when the drive-reported resolution differs from the datasheet encoder CPR because of internal interpolation, electronic gearing, quadrature decoding, or application scaling inside the drive. The correct procedure is to identify the effective drive-side unit first, then identify the mechanical ratio separately.

The practical rule is:

- use `counts_per_rev` to match the actual raw position delta for one drive-side revolution
- use `gear_ratio` to match how many drive-side revolutions correspond to one controlled-axis revolution
- avoid hiding an internal-scaling error inside the mechanical ratio field

For the turret azimuth mast, this matters directly because an incorrect ratio produces large angular mismatch between commanded turret degrees and real mast movement. For the repeated MYACTUATOR axes, the same principle applies when an actuator has an external linkage or when the vendor command unit already represents output-side angle.

8.11 Nanotec Commissioning Procedure

Nanotec commissioning is the controlled process used to verify and, when required, write drive object-dictionary parameters before the runtime is allowed to command motion. It should be performed on a bench setup before field deployment.

8.11.1 Preconditions

Before writing any drive parameter:

- mechanically unload the axis where possible
- confirm emergency stop access
- confirm CAN bitrate and node ID
- save the original object-dictionary values
- use `candump` or an equivalent CAN monitor
- keep the STC runtime stopped during persistent OD changes

8.11.2 Objects that may be written

The normal STC runtime sends control and target values through PDOs. Persistent object-dictionary writes belong to commissioning.

Typical commissioning writes are:

- `1017h` heartbeat producer time, used for node health supervision
- `1400h`, `1401h`, `1402h` RPDO communication parameters
- `1600h`, `1601h`, `1602h` RPDO mapping parameters
- `1800h`, `1801h`, `1802h` TPDO communication parameters
- `1A00h`, `1A01h`, `1A02h` TPDO mapping parameters
- vendor save and restore objects when persistent storage is required by the drive profile

Runtime values such as `6040h`, `6060h`, `607Ah`, `6081h`, and `6042h` are sent during operation and do not represent commissioning state by themselves.

8.11.3 Recommended write sequence

A safe PDO remapping sequence is:

1. place the node in pre-operational state
2. disable the PDO being remapped through its communication parameter
3. clear the mapping count to zero
4. write the mapped object entries in the required order
5. write the final mapping count
6. restore the PDO communication parameter and transmission behavior

7. restart the node or return it to operational state
8. verify the produced PDOs with `candump`
9. run a small bounded motion test

The expected final mapping is the layout shown in Table 8.1. Any persistent save should happen only after the mapping has been verified.

8.11.4 Revert procedure

A safe revert is:

1. stop the STC runtime
2. return the node to pre-operational state
3. write back the saved original communication and mapping objects
4. power-cycle or reset the node
5. verify heartbeat, TPDOs, and SDO identity reads
6. rerun a bounded no-load motion test

If the original values are unavailable, restore the vendor default mapping through the vendor tool or the drive restore mechanism, then reapply the STC mapping from the documented baseline.

8.12 MYACTUATOR Commissioning Procedure

MYACTUATOR commissioning focuses on node identity, feedback validity, zero position, optional protection settings, and active reply behavior.

8.12.1 Preconditions

Before operating a MYACTUATOR axis:

- confirm node ID in the range supported by the library
- confirm CAN bitrate
- confirm model identity and firmware date
- read status registers and verify voltage, temperature, and error flags

- verify that stop and shutdown commands are effective
- keep mechanical travel clear during zeroing and first motion tests

8.12.2 Commands used during commissioning

The main commissioning functions are:

- `myact_read_model()` for model identification
- `myact_read_version_date()` for firmware build date
- `myact_canid_read()` and `myact_canid_write()` for CAN-ID service
- `myact_read_status1()`, `myact_read_status2()`, and `myact_read_status3()` for health and feedback checks
- `myact_write_zero_offset()` and `myact_write_zero_from_current()` for zero alignment
- `myact_accel_write_rom()` for acceleration limits
- `myact_pid_write_ram()` and `myact_pid_write_rom()` for PID tuning
- `myact_set_comm_protect_ms()` for communication protection
- `myact_active_reply_config()` for active periodic replies
- `myact_set_positive_limit_raw()` and `myact_set_negative_limit_raw()` for device-side limit support

Values written to ROM should be treated as persistent commissioning state. Values written to RAM are useful for testing and tuning because they do not require permanent commitment.

8.12.3 Recommended MYACTUATOR bring-up sequence

A controlled bring-up sequence is:

1. initialize the transport callbacks and the node context
2. read current CAN ID, model, firmware date, and runtime
3. read status registers and confirm that no error flags are active
4. issue `stop` and verify that the device responds
5. read angle and encoder feedback
6. set zero only after mechanical reference is confirmed

7. test a small absolute or incremental move at low speed
8. test speed command with a short watchdog window
9. enable active replies only after the single-command path is verified
10. record final node ID, zero offset, limits, acceleration, PID values, and firmware information

8.12.4 Revert procedure

A safe revert is:

1. send stop
2. lock or disable output if the actuator supports the required state
3. restore saved PID, acceleration, zero offset, CAN ID, and communication protection values
4. disable active replies if they interfere with bench diagnostics
5. power-cycle the actuator
6. verify identity, status, angle feedback, and stop behavior

8.13 Bench Calibration Narrative

Bench calibration should prove three facts before integration:

- one commanded engineering-unit movement produces the expected physical movement
- feedback units are interpreted correctly
- stop and fault paths work before larger moves are attempted

For the Nanotec azimuth mast path, the most important check is counts-per-output-degree. The procedure is:

1. command a small safe movement
2. measure physical output movement
3. read raw position feedback
4. compute effective counts per output degree
5. compare the result to `counts_per_rev` multiplied by `gear_ratio`

6. adjust only the field that represents the observed mismatch

For MYACTUATOR SUT and mirror axes, the equivalent check depends on the command family:

- for standard position commands, compare `0.01 deg` command units with physical output movement
- for motion mode, compare packed radian position and velocity with the measured output
- for external linkages, validate the axis-level mechanical ratio separately from the actuator internal unit

The calibration record should include the device model, node ID, CAN bitrate, firmware identity, zero offset, measured movement, raw feedback delta, configured limits, and final conversion constants.

8.14 Runtime Safety Rules for Motion

The motion layer follows these runtime safety rules:

- ordinary motion is accepted only in `OPER`
- speed streaming requires periodic refresh from the Control Unit
- leaving `OPER` clears active stream state
- entering `FAIL` schedules quick stop across relevant axes
- unavailable or faulted axes are skipped by grouped commands
- command limits are applied before drive-unit conversion
- saturation is preserved as telemetry and diagnostic information
- backend I/O is owned by the CAN worker

These rules allow the same high-level server behavior to drive both motor families while keeping protocol-specific details inside the corresponding low-level library and backend path.

8.15 Chapter Summary

The STC motion layer uses a standards-based DS402 path for the Nanotec azimuth mast motor and a vendor CAN path for the MYACTUATOR SUT

and mirror actuators. The server sends all commands in engineering units, the backend applies limits and conversion, and the low-level libraries handle the actual bus protocol.

The most important implementation principle is separation of concerns: STC state and command semantics stay in the server, axis scaling stays in the configuration model, device protocol behavior stays in the low-level libraries, and persistent drive changes are restricted to controlled commissioning procedures.

Chapter 9

Simulation, User Interfaces, and Verification

9.1 Purpose of this Chapter

This chapter documents the human-facing and verification-enabling interfaces around the STC runtime. The previous chapters described the server, protocol, state machine, and motion layer. This chapter focuses on how those components are exercised without full field deployment: the simulator backend, the Qt diagnostic GUI, the CLI tools, motion demonstration programs, unit tests, stress tests, and virtual-CAN workflows.

The goal is to make development and integration repeatable. Each tool has a specific role. The simulator enables software motion behavior without drives. The GUI supports operator-style inspection. The CLI tools provide deterministic protocol injection and telemetry decoding. The tests verify the core logic and concurrency assumptions.

9.2 Simulator Backend

The simulator backend implements the same `motion_backend.h` contract as the real CAN backend. This is the main reason it is useful: the server can execute the same command, state, action, watchdog, and telemetry logic while the hardware-facing behavior is replaced by a deterministic software motor model.

The simulator is implemented in C++ and linked into the C server through

the backend interface. The implementation uses `std::thread`, `std::mutex`, `std::atomic`, fixed-size `std::array` storage, and the shared C motion-backend structures. This keeps the simulator convenient to implement while preserving the same C ABI (Application Binary Interface) seen by `stc_server`.

The simulator is selected at build time through:

```
cmake -S . -B build -DUSE_SIMULATOR=ON
cmake --build build -j
```

Listing 9.1: Simulator build profile

When `USE_SIMULATOR=ON`, CMake links `stc_server` against the static simulator library. When `USE_SIMULATOR=OFF`, the server uses the real hardware CAN backend.

9.2.1 Simulation details

The backend starts one simulator context containing 18 simulated axes. Each axis stores its axis configuration, a `MotorModel`, the active target type, the last speed-command timestamp, the latest receive timestamp, and a simulated mode display.

The backend integrator runs at a fixed 10 ms period. Position targets are integrated through a bounded velocity and acceleration model. Speed targets are tracked directly with acceleration limiting. Configured travel limits and speed limits are applied by the motor model. Simulated raw position and raw velocity are converted using the same axis scaling helpers used by the server model.

The `MotorModel` represents one axis with position, velocity, acceleration, minimum position, maximum position, maximum velocity, and maximum acceleration. Position control uses a trapezoidal-style profile that accelerates, cruises, brakes, and snaps to target when close enough. Velocity control tracks a signed target speed while respecting the configured maximum velocity and acceleration.

The simulator also implements the same stream-watchdog idea as the runtime. For each speed-controlled axis, it stores `last_speed_cmd_ms`. If the configured `speed_watchdog_ms` expires, the simulator clears speed mode, sets the velocity target to zero, clears the mode display, and stops updating that axis as an active stream.

Quick stop is modeled by resetting the affected simulated axis at its current position with zero velocity. The simulator also clears the position target,

speed target, mode display, and raw velocity. This gives the server a realistic software response for stop-path testing, even though no real drive state transition occurs.

The simulator produces backend snapshots that look like hardware snapshots from the server point of view. At startup it marks all configured axes available, sets `can_ok = 1`, and reports `STC_AXIS_COUNT` connected devices. During motion it reports:

- `status_word = 0x1237` for operation-enabled style feedback
- `mode_display = PD4C_MODE_PROFILE_POSITION` during position moves
- `mode_display = PD4C_MODE_PROFILE_VELOCITY` during speed commands
- `mode_display = 0` when the axis is stopped or inactive
- `last_rx_ms` updated during active simulated motion

These values are adequate for validating server logic, telemetry construction, state gating, watchdog behavior, and GUI visualization.

9.2.2 Simulator limitations

The simulator is a control-software verification tool. It cannot reproduce all hardware effects. The main limitations are:

- no real CAN arbitration, bus load, electrical errors, or adapter behavior
- no true DS402 state transitions inside a drive
- no real SDO latency, PDO timing jitter, heartbeat loss, or EMCY emission
- no motor current, voltage, thermal behavior, backlash, compliance, or mechanical friction
- no vendor-specific actuator error registers
- no calibration proof for counts, ratios, zero position, or real motion envelope

For this reason, simulator tests are necessary for software regression and unsafe to treat as final hardware acceptance. Hardware commissioning, low-speed bounded motion, stop-path validation, and CAN trace inspection are still required before field use.

9.3 Qt Diagnostic GUI

The optional Qt diagnostic GUI is the main human-facing engineering tool. It is built only when Qt6 Core, Gui, Widgets, and Network packages are available.

```
cmake -S . -B build_gui -DBUILD_DIAG_GUI=ON
cmake --build build_gui -j
./build_gui/diag_gui_qt
```

Listing 9.2: Qt GUI build profile

The GUI uses the same `stc_icd` codec as the server tools. It opens a TCP client connection to the STC command port, binds UDP sockets for TLMH and TLML, builds command frames, parses response frames, and displays decoded telemetry.

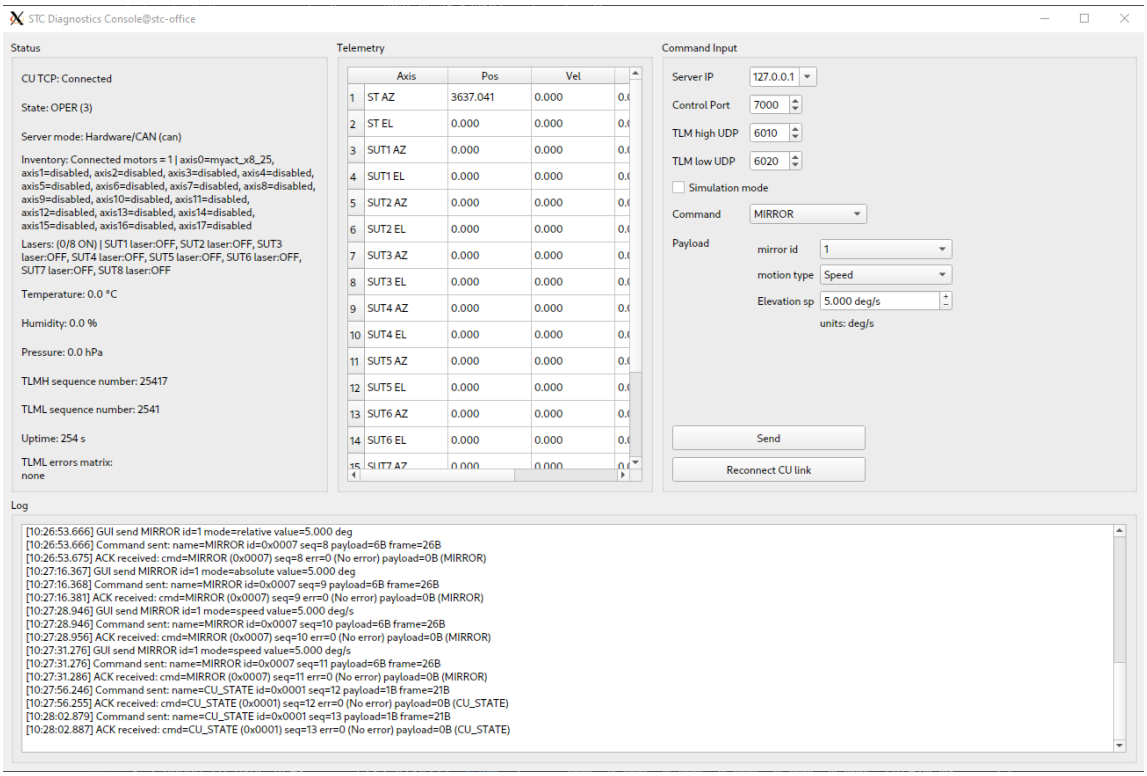


Figure 9.1: Graphical User Interface

9.3.1 Feature coverage

The GUI covers the following engineering functions:

- selecting server IP, command port, telemetry ports, and controller identifier
- sending ICD command families such as `CU_STATE`, `SELFLEV`, `IBIT`, `STORAGE`, `LASER`, `TURRET`, `MIRROR`, `SUT`, `RESET`, `MANUALPOS`, `POSMODE`, and information requests
- displaying `ACK` and `NACK` responses with decoded error codes
- listening to TLMH and updating per-axis position, velocity, and acceleration fields
- listening to TLML and displaying state, uptime, environmental values, position context, laser state, and diagnostic errors
- supporting a local GUI simulation mode for interface testing without a server
- supporting reconnect and telemetry socket rebinding during bench work

The GUI is therefore an ICD client, not a privileged bypass around the server. Commands sent from the GUI still pass through the same TCP protocol, validation path, state gates, action queue, and backend ownership model as commands from the Control Unit.

9.3.2 Relationship to diagnostics

The GUI primarily consumes TLML diagnostics. The optional local diagnostics JSON endpoint belongs to runtime observability and should remain read-only. When this endpoint is enabled, the GUI can use it to show detailed local health information such as queue depth, jitter counters, command latency, per-axis stale age, and degraded causes.

9.4 CLI Tools

The CLI tools are the preferred deterministic tools for protocol-level integration work. They are simpler than the GUI and are suitable for scripts, automated tests, SSH sessions, and repeatable bench procedures.

9.4.1 icd_tx_tcp

`icd_tx_tcp` builds binary ICD command frames and sends them to a running server. It is used for:

- checking state transitions
- injecting motion commands
- validating `ACK` and `NACK` behavior
- testing malformed frames through negative test modes
- scripting motion acceptance tests

Example use:

```
./build/icd_tx_tcp 127.0.0.1 7000 cu_state 2
./build/icd_tx_tcp 127.0.0.1 7000 cu_state 3
./build/icd_tx_tcp 127.0.0.1 7000 turret abs 10.0 0.0
./build/icd_tx_tcp 127.0.0.1 7000 turret speed 5.0 0.0
./build/icd_tx_tcp 127.0.0.1 7000 bad_payload
```

Listing 9.3: CLI command examples

9.4.2 tlm_rx_udp

`tlm_rx_udp` receives and decodes TLMH and TLML telemetry. It is used to verify that accepted commands produce the expected telemetry effects and that state, health, laser, and diagnostic fields remain coherent.

Example use:

```
./build/tlm_rx_udp
./build/tlm_rx_udp --ifname eth0
./build/tlm_rx_udp --raw
./build/tlm_rx_udp --verbose
```

Listing 9.4: CLI telemetry examples

In practice, `icd_tx_tcp` and `tlm_rx_udp` are used together. One terminal sends deterministic commands. A second terminal observes the corresponding telemetry result.

9.5 Motion Demonstration Programs

The repository contains two motion-demonstration paths. They should be treated as integration acceptance tests for motion behavior, not as ordinary user applications.

9.5.1 Scripted ICD turret demo

The shell demo `turret_motion_demo.sh` is copied by CMake into the build directory as `build/turret_motion_demo`. It drives the server through `icd_tx_tcp`, so it validates the full command path from TCP command injection to server action handling.

The default envelope is:

- home position at `0 deg`
- azimuth minimum at `-140 deg`
- azimuth maximum at `+140 deg`
- maximum load speed at `25 deg/s`
- fine absolute-position step size at `0.1 deg`

The sequence is:

1. enter `OPER`
2. command home
3. hold center
4. move toward the negative limit by speed command
5. stop and hold
6. move toward the positive limit by speed command
7. stop and hold
8. return home
9. run a fine absolute-position step sequence
10. return home

The script installs a `SIGINT` and `SIGTERM` trap. On interruption, it sends a zero-speed command, waits briefly, commands home, and exits with the

interrupted status. This makes the script suitable for bench acceptance because the operator can use `CTRL+C` and still trigger a controlled stop and home action.

```
./build/stc_server --config dev/config/stc_default.ini

./build/turret_motion_demo 127.0.0.1 7000
```

Listing 9.5: Scripted turret motion demo

9.5.2 Standalone Nanotec C demo

The standalone C demo `turret_motion_demo.c` talks directly to the Nanotec low-level library. It is useful during drive and azimuth-axis commissioning because it removes the server and ICD layers from the test path and focuses on DS402, scaling, motion limits, and safe stop.

The demo contains several calibration-specific behaviors:

- it reads the active drive position resolution from `608F` when available
- it separates drive internal resolution, nominal gearbox ratio, and extra mechanical ratio
- it can widen `607D` software position limits when the corrected $\pm 140^\circ$ range requires it
- it logs objects such as `60A8`, `60A9`, `608F`, `6091`, and `6092`
- it keeps unit-stack writes disabled by default
- it performs `CTRL+C` safety stop and home behavior

Example build and run:

```
gcc -O2 -Wall -Wextra -std=c11 -D_DEFAULT_SOURCE
   -D_POSIX_C_SOURCE=200809L \
  -Idev/include \
  dev/tests/turret_motion_demo.c dev/src/pd4c.c -lm \
  -o build/turret_motion_demo_c

./build/turret_motion_demo_c can0 1
```

Listing 9.6: Standalone C motion demo

This demo should be run only with a safe mechanical setup, visible emergency stop access, and a known node ID. It is an acceptance test for the

azimuth motor path and a calibration aid for the ratio and drive-scaling model documented in Chapter 8.

9.6 Test Suite

The active CMake build defines focused unit, integration, and stress targets. The important point is that tests are organized around the same architectural seams used by the runtime: actions, state, diagnostics, ICD encoding, configuration, logging, and worker concurrency.

Table 9.1: Main verification targets

Target	**Verification Role**
`test_actions`	Verifies action construction, FIFO queue behavior, eventfd wakeup, and keep-latest stream-speed coalescing.
`stc_state_test`	Verifies state transitions, invalid transitions, movement and stream predicates, DS402 fault detection, CAN timeout detection, and BIT event mapping.
`stc_diag_test`	Verifies diagnostic-code packing and the baseline motor-status flag model.
`test_diag_state_integration`	Verifies integration between diagnostic outcomes and state-machine behavior.
`test_icd_codec`	Verifies command, response, TLMH, and TLML pack and parse logic. The final version shall assert the `motor_status[18]` TLML layout from Appendix A.
`stc_config_test`	Verifies configuration parsing, rejection of typoed keys and sections, deprecated network-key compatibility, and strict stale-policy parsing.
`test_log_concurrency`	Verifies that the logging path tolerates concurrent writers.
`test_can_worker_concurrency`	Compiles the server into the test harness, starts the CAN worker, injects large action traffic, mutates shared telemetry fields, and probes action dispatch behavior under concurrency.

A normal verification run after building is:

```
./build/test_actions
./build/stc_state_test
./build/stc_diag_test
./build/test_diag_state_integration
./build/test_icd_codec
./build/stc_config_test
./build/test_log_concurrency
./build/test_can_worker_concurrency
./build/test_can_worker_concurrency_tsan
```

Listing 9.7: Regression test run

9.7 Virtual CAN Strategy

Linux `vcan0` is used when a developer needs SocketCAN behavior without physical CAN hardware. It is especially useful for checking build profiles, raw socket permissions, preflight behavior, CAN-interface selection, and receive or transmit tooling.

A minimal setup is:

```
sudo modprobe vcan
sudo ip link add dev vcan0 type vcan
sudo ip link set vcan0 up
ip -details link show vcan0
```

Listing 9.8: Manual vcan setup

The repository also provides a helper script:

```
sudo ./scripts/vcan_setup.sh
```

Listing 9.9: Repository vcan helper

Two workflows are recommended.

9.7.1 Software behavior workflow

Use the simulator backend when the goal is to validate server behavior, ICD commands, telemetry, state gates, and watchdog logic.

```
cmake -S . -B build -DUSE_SIMULATOR=ON
cmake --build build -j
```

```
3
4 ./build/stc_server --config dev/config/stc_default.ini
5 ./build/tlm_rx_udp --verbose
6 ./build/icd_tx_tcp 127.0.0.1 7000 cu_state 3
7 ./build/icd_tx_tcp 127.0.0.1 7000 turret abs 10.0 0.0
```

Listing 9.10: Simulator workflow

This workflow provides meaningful simulated motion telemetry because the simulator backend owns a software motor model.

9.7.2 SocketCAN workflow

Use `vcan0` with the hardware backend when the goal is to validate raw SocketCAN access, service permissions, CAN socket binding, and bus-monitor tooling.

```
1 cmake -S . -B build_hwcan -DUSE_SIMULATOR=OFF
     -DBUILD_DIAG_GUI=OFF
2 cmake --build build_hwcan -j
3
4 candump -tz -cae vcan0
5
6 ./build_hwcan/stc_server --can vcan0 --config
     dev/config/stc_default.ini
```

Listing 9.11: Hardware-backend build against vcan0

This workflow validates the Linux CAN path. It does not create real drive responses by itself. For full receive-side behavior, a test or helper process must inject expected CAN frames into `vcan0`. For motion behavior without injected drive frames, use the simulator backend.

9.8 Verification Boundaries

The verification interfaces cover different confidence levels:

- unit tests prove local logic such as encoding, parsing, transitions, queue behavior, and configuration validation
- simulator tests prove server-level behavior across the ICD, state gates, action queue, backend contract, watchdogs, and telemetry
- `vcan0` tests prove Linux SocketCAN access and CAN tooling workflows

- GUI tests prove operator-facing visibility and manual command paths
- motion demos prove bounded integration behavior on real motion paths
- hardware commissioning proves the physical control envelope, scaling, stop behavior, and real bus behavior

A successful simulator or `vcan0` run should be treated as software readiness for hardware bench testing. Final acceptance still requires low-speed hardware validation, CAN trace inspection, axis calibration, safe-stop verification, and telemetry review under the real mechanical configuration.

References

Implementation references

STC runtime and configuration: dev/src/ and dev/include/.

Software architecture: manuals/ARCHITECTURE.md.

Wire protocol: stc_icd.h, stc_icd.c, and manuals/communication_protocol.md.

Operational state transitions: stc_state.h and stc_state.c.

Configuration examples: dev/config/.

Build, deployment, and verification: manuals/SETUP.md and deploy/.

External references

[9] ISO 11898-1: Road Vehicles, Controller Area Network, Part 1, Data Link Layer and Physical Signalling.

[10] ISO 11898-2: Road Vehicles, Controller Area Network, Part 2, High-Speed Medium Access Unit. Latest applicable edition.

[11] CAN in Automation. *CiA 301 Version 4.2.0, CANopen Application Layer and Communication Profile*. CAN in Automation, Nuremberg, Germany, 2011.

[12] CAN in Automation. *CiA 402-1 Version 5.0.0, CANopen Device Profile for Drives and Motion Control, Part 1, General Definitions*. CAN in Automation, Nuremberg, Germany, 2023.

[13] CAN in Automation. *CiA 402-2 Version 5.0.0, CANopen Device Profile for Drives and Motion Control, Part 2, Operation Modes and Application Data*. CAN in Automation, Nuremberg, Germany, 2024.

[14] CAN in Automation. *CiA 402-3 Version 5.0.0, CANopen Device Profile for Drives and Motion Control, Part 3, PDO Mapping*. CAN in Automation, Nuremberg, Germany, 2024.

[15] CAN in Automation. CANopen: The Standardized Embedded Network. Online reference. Accessed 2026-05-22.

[16] CAN in Automation. Process Data Object Protocol. CANopen PDO protocol reference. Accessed 2026-05-22.

[17] CAN in Automation. Service Data Object Protocol. CANopen SDO protocol reference. Accessed 2026-05-22.

[18] CAN in Automation. Network Management. CANopen NMT and node-control protocol reference. Accessed 2026-05-22.

[19] IEC 61800-7-201, Adjustable Speed Electrical Power Drive Systems, Generic Interface and Use of Profiles for Power Drive Systems, Profile Type 1 Specification.

[20] IEC 61800-7-301, Adjustable Speed Electrical Power Drive Systems, Generic Interface and Use of Profiles for Power Drive Systems, Mapping of Profile Type 1 to Network Technologies.

[21] IEEE Std 754-2019, IEEE Standard for Floating-Point Arithmetic, 2019.

[22] Nanotec Electronic GmbH & Co. KG. *PD4-C Plug & Drive Motor Controller Technical Manual*. Nanotec Electronic GmbH & Co. KG, Feldkirchen, Germany.

[23] Nanotec Electronic GmbH & Co. KG. *PD4-C CANopen Object Dictionary and Communication Manual*. Nanotec Electronic GmbH & Co. KG, Feldkirchen, Germany.

[24] Nanotec Electronic GmbH & Co. KG. *PD6-C Plug & Drive Motor Controller Technical Manual*. Nanotec Electronic GmbH & Co. KG, Feldkirchen, Germany.

[25] Nanotec Electronic GmbH & Co. KG. *PD6-C CANopen Object Dictionary and Communication Manual*. Nanotec Electronic GmbH & Co. KG, Feldkirchen, Germany.

[26] Nanotec Electronic GmbH & Co. KG. *NanoJ and Plug & Drive Studio Documentation*. Nanotec Electronic GmbH & Co. KG, Feldkirchen, Germany.

[27] MYACTUATOR. *RMD Series CAN Protocol Manual*. MYACTUATOR. Vendor CAN command, reply, status, motion-mode, PID, acceleration, zero-offset, and communication-protection reference.

[28] MYACTUATOR. *X8-25 Actuator Datasheet and User Manual*. MYACTUATOR. X8-25 motor, gearbox, encoder, electrical, mechanical, and CAN-control reference.

[29] MYACTUATOR. *V4-X8-120 Actuator Datasheet and User Manual*. MYACTUATOR. X8-120 actuator reference for SUT motion-axis integration.

[30] MYACTUATOR. *L-9025 Actuator Datasheet and User Manual*. MYACTUATOR. L-series actuator reference for mirror-axis integration.

[31] PEAK-System Technik GmbH. *PCAN-USB User Manual*. PEAK-System Technik GmbH. CAN adapter installation, driver, electrical, and diagnostic reference.

[32] PEAK-System Technik GmbH. *PCAN-Basic API Documentation*. PEAK-System Technik GmbH. Host-side CAN communication API reference for bench diagnostics and adapter validation.

[33] Noxant. *NoxCore-275 MWIR Camera Module Datasheet*. Noxant. Cooled MWIR camera module reference for payload, optical, Ethernet, serial-control, and video-streaming integration.

[34] Noxant. *NoxCore Family User Manual and Software Interface Documentation*. Noxant. Camera-family control, configuration, video output, serial command, and network-streaming reference.

[35] Chiaravalli Group. *Industrial Gearboxes, Gear Motors, and Transmission Products Documentation*. Chiaravalli Group. Mechanical transmission reference for the self-levelling subsystem.

[36] PINE64. ROCKPro64 Documentation. Official board documentation. Accessed 2026-05-22.

[37] PINE64. ROCKPro64 Board Features. Official board features, RK3399 processor, memory, PCIe, Ethernet, USB, and expansion-interface reference. Accessed 2026-05-22.

[38] Rockchip Electronics Co., Ltd. *RK3399 Datasheet*. Rockchip Electronics Co., Ltd. RK3399 processor electrical, peripheral, and SoC feature reference.

[39] Rockchip Electronics Co., Ltd. *RK3399 Technical Reference Manual*. Rockchip Electronics Co., Ltd. RK3399 CPU, memory, interrupt, timer, peripheral, and low-level hardware reference.

[40] Manjaro ARM Project. Manjaro ARM Documentation. Target Linux distribution and package-management reference for RockPro64 deployment. Accessed 2026-05-22.

[41] Linux Kernel Project. SocketCAN, Controller Area Network. Linux Kernel networking documentation. Accessed 2026-05-22.

[42] Linux Kernel Project. Linux CAN Networking Headers and Userspace API. Kernel UAPI reference for `linux/can.h`, `linux/can/raw.h`, CAN RAW sockets, frame layout, and receive filters. Accessed 2026-05-22.

[43] Linux Manual Pages Project. sched(7), Linux Scheduling API. Scheduling policies, priorities, `SCHED_FIFO`, `SCHED_RR`, and `SCHED_OTHER`. Accessed 2026-05-22.

[44] Linux Manual Pages Project. capabilities(7), Linux Capabilities. Linux process capability reference for `CAP_SYS_NICE`, `CAP_NET_RAW`, and service privilege reduction. Accessed 2026-05-22.

[45] Linux Manual Pages Project. socket(2), bind(2), poll(2), eventfd(2), clock_nanosleep(2), and clock_gettime(2). Linux system-call references used by the STC runtime. Accessed 2026-05-22.

[46] systemd Project. systemd.exec Manual. Service execution environment, capabilities, resource limits, CPU affinity, scheduling policy, and memory-lock configuration. Accessed 2026-05-22.

[47] systemd Project. systemd.service Manual. Service unit lifecycle, restart behavior, pre-start checks, and managed daemon deployment reference. Accessed 2026-05-22.

[48] iproute2 Project. iproute2 User-Space Networking Utilities. Linux interface configuration reference for `ip link`, `can0`, and `vcan0` setup. Accessed 2026-05-22.

[49] linux-can Project. can-utils. SocketCAN diagnostic utilities including `candump`, `cansend`, `cangen`, and `cansniffer`. Accessed 2026-05-22.

[50] J. Postel. RFC 791, Internet Protocol. IETF RFC, 1981.

[51] J. Postel. RFC 768, User Datagram Protocol. IETF RFC, 1980.

[52] W. Eddy. RFC 9293, Transmission Control Protocol. IETF RFC, 2022.

[53] D. Mills, J. Martin, J. Burbank, and W. Kasch. RFC 5905, Network Time Protocol Version 4, Protocol and Algorithms Specification. IETF RFC, 2010.

[54] R. Braden. RFC 1122, Requirements for Internet Hosts, Communication Layers. IETF RFC, 1989.

[55] S. Bradner. RFC 2119, Key Words for Use in RFCs to Indicate Requirement Levels. IETF RFC, 1997.

[56] B. Leiba. RFC 8174, Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words. IETF RFC, 2017.

[57] NMEA 0183 Standard for Interfacing Marine Electronic Devices. Latest applicable edition.

[58] TIA-485-A, Electrical Characteristics of Generators and Receivers for Use in Balanced Digital Multipoint Systems.

[59] National Geospatial-Intelligence Agency. Department of Defense World Geodetic System 1984, Its Definition and Relationships with Local Geodetic Systems. Technical Report TR8350.2, National Geospatial-Intelligence Agency.

[60] ISO 8601-1, Date and Time, Representations for Information Interchange. Latest applicable edition.

[61] ISO/IEC 9899:2018, Programming Languages, C, 2018.

[62] ISO/IEC 14882:2017, Programming Languages, C++, 2017.

[63] IEEE Std 1003.1-2024, POSIX Base Specifications, Issue 8, 2024.

[64] Kitware. CMake Reference Documentation. CMake build-system, target, generator, and configuration reference. Accessed 2026-05-22.

[65] GNU Project. GCC Online Documentation. GNU Compiler Collection reference for C and C++ compilation, diagnostics, and command-line options. Accessed 2026-05-22.

[66] GNU Project. GNU Make Manual. Build automation reference for make-based workflows and generated build rules. Accessed 2026-05-22.

[67] Ninja Build. Ninja Manual. Low-overhead build executor reference for generated CMake builds. Accessed 2026-05-22.

[68] GNU Project. GDB Documentation. Debugger reference for native Linux C and C++ target debugging. Accessed 2026-05-22.

[69] LLVM Project. ThreadSanitizer Documentation. Data-race detection reference used for concurrency-oriented test builds. Accessed 2026-05-22.

[70] Valgrind Developers. Valgrind User Manual. Runtime memory, leak, and dynamic-analysis reference. Accessed 2026-05-22.

[71] The Qt Company. Qt 6 Documentation. Qt Core, Qt GUI, Qt Widgets, Qt Network, and Qt Test documentation. Accessed 2026-05-22.

[72] The Qt Company. Qt Widgets Module Documentation. Widget-based desktop GUI reference for diagnostic interface construction. Accessed 2026-05-22.

[73] The Qt Company. Qt Network Module Documentation. TCP, UDP, socket, datagram, and host-address reference for GUI communication with the STC daemon. Accessed 2026-05-22.

[74] The Qt Company. Qt Test Module Documentation. Unit and integration test support for Qt-based tooling. Accessed 2026-05-22.

[75] X.Org Foundation. X Window System Documentation. X11 forwarding and remote GUI-display reference for SSH-based target workflows. Accessed 2026-05-22.

[76] OpenSSH Project. OpenSSH Manual Pages. SSH client, server, trusted X11 forwarding, and remote development connection reference. Accessed 2026-05-22.

[77] IEEE Std 1012, IEEE Standard for System, Software, and Hardware Verification and Validation.

[78] IEEE Std 828, IEEE Standard for Configuration Management in Systems and Software Engineering.

[79] IEEE Std 830, IEEE Recommended Practice for Software Requirements Specifications.

[80] ISO/IEC/IEEE 29148, Systems and Software Engineering, Life Cycle Processes, Requirements Engineering.

[81] ISO/IEC/IEEE 12207, Systems and Software Engineering, Software Life Cycle Processes.

[82] Object Management Group. Unified Modeling Language, UML Version 2.5.1. Formal modeling reference for state-machine and structural diagrams. Accessed 2026-05-22.

[83] NASA. *NASA Software Safety Guidebook*. NASA. Software safety-process reference for hazard-driven control-software design.

[84] Michael Barr. *Embedded C Coding Standard*. Barr Group. Embedded C design, clarity, and defensive-programming reference.

[85] Philip Koopman. *Better Embedded System Software*. Drumnadrochit Education. Embedded-system safety, watchdog, testing, robustness, and field-failure engineering reference.

[86] Alan Burns and Andy Wellings. *Real-Time Systems and Programming Languages*. Addison-Wesley. Real-time scheduling, concurrency, and deterministic software behavior reference.

[87] Giorgio C. Buttazzo. *Hard Real-Time Computing Systems*. Springer. Scheduling theory and real-time control-system implementation reference.

[88] Andrew S. Tanenbaum and Herbert Bos. *Modern Operating Systems*. Pearson. Operating-system process, scheduling, concurrency, and I/O reference.

[89] W. Richard Stevens, Bill Fenner, and Andrew M. Rudoff. *UNIX Network Programming, Volume 1, The Sockets Networking API*. Addison-Wesley. Socket, TCP, UDP, polling, and network-programming reference.

[90] Michael Kerrisk. *The Linux Programming Interface*. No Starch Press. Linux system programming, process control, sockets, timers, signals, and threading reference.

[91] Leslie Lamport. *LaTeX: A Document Preparation System.* Addison-Wesley, 2 edition.

[92] Frank Mittelbach, Michel Goossens, Johannes Braams, David Carlisle, and Chris Rowley. *The LaTeX Companion.* Addison-Wesley.

[93] Overleaf. Overleaf LaTeX Documentation. Online LaTeX authoring, project organization, bibliography, figures, tables, and compilation reference. Accessed 2026-05-22.

[94] TeX Users Group. TeX Live Documentation. LaTeX compiler, package, font, and document-build environment reference. Accessed 2026-05-22.

[95] GNU Project. GNU diffutils Manual. Text comparison and documentation-review support reference. Accessed 2026-05-22.

[96] Git Project. Git Documentation. Version-control reference for source, configuration, deployment, and documentation traceability. Accessed 2026-05-22.

Appendix A

ICD Byte-Level Reference

A.1 Purpose

This appendix defines the byte-level CU–STC communication contract. Chapter 6 describes the operational meaning of the ICD, while this appendix records the exact constants, frame layouts, payload sizes, command identifiers, telemetry formats, and diagnostic encoding.

A.2 Global Constants and Encoding

Table A.1: Global ICD constants

Item	Value	Meaning
Protocol version	`0x01`	Current wire-protocol version
Command magic	`TCMD`	TCP command frame
Response magic	`TRSP`	TCP response frame
High telemetry magic	`TLMH`	UDP high-rate telemetry frame
Low telemetry magic	`TLML`	UDP low-rate telemetry frame
Axis count	`18`	Number of configured motion axes
Maximum payload	2048 bytes	Maximum command or response payload accepted by the active ICD codec
Maximum diagnostic entries	`64`	Maximum TLML diagnostic-code count
Default TCP port	`7000`	Command and response channel
Default TLMH port	`6010`	High-rate telemetry channel
Default TLML port	`6020`	Low-rate telemetry channel
Default TLMH period	`10 ms`	Nominal 100 Hz publication
Default TLML period	`100 ms`	Nominal 10 Hz publication
Telemetry destination	Active TCP client IP	UDP unicast while CU is connected
Controller role	Control Unit	Single authoritative external controller

Table A.2: Primitive encoding rules

Type	Size	Encoding
`uint8_t`	1 byte	Unsigned integer
`uint16_t`	2 bytes	Little-endian unsigned integer
`uint32_t`	4 bytes	Little-endian unsigned integer
`uint64_t`	8 bytes	Little-endian unsigned integer
`float`	4 bytes	IEEE-754 single precision, little-endian storage
`double`	8 bytes	IEEE-754 double precision, little-endian storage
ASCII payload	variable	Raw bytes with no required terminator

A.3 Common Enumerations

Table A.3: Response result values

Name	Value	Meaning
ACK	0x00	Command accepted
NACK	0x01	Command rejected

Table A.4: Response error codes

Code	Name	Meaning
0x0000	NoError	No error
0x0001	ClientUnknown	Unknown or unauthorized controller source
0x0002	ProtocolMismatch	Protocol version, magic, or framing mismatch
0x0003	SyntaxError	Malformed command or wrong payload size
0x0004	ValueOutOfRange	Invalid value, invalid identifier, or limit violation
0x0005	NotAllowedInState	Command rejected by the current STC state
0x0006	InsufficientResources	Resource shortage or allocation failure
0x0007	Timeout	Timeout condition
0x0008	CommandNotImplemented	Recognized command unavailable in the active profile
0x00FF	Generic	Generic failure

Table A.5: Motion type, position mode, and state identifiers

Enum	Value	Name	Meaning
Motion type	0	Relative	Offset in degrees
Motion type	1	Absolute	Absolute target in degrees
Motion type	2	Speed	Speed in degrees per second
Position mode	0	GPS	Use GPS position source
Position mode	1	Manual	Use manual position and heading
STC state	0	INIT	Initialization
STC state	1	INIT_DONE	Initialization completed
STC state	2	SAFE	Safe state
STC state	3	OPER	Operational state
STC state	4	SELF_LEVELLING	Self-levelling active
STC state	5	IBIT_RUNNING	Interactive BIT running
STC state	6	STORE	Storage state
STC state	0xFF	FAIL	Failure state

A.4 TCP Frames

Table A.6: TCP command frame layout

Offset	Field	Size	Description
0	magic	4	ASCII TCMD
4	protocol_version	1	Current value 0x01
5	controller_id	1	Control Unit identifier
6	cmd_id	2	Command identifier
8	seq	2	Command sequence number
10	timestamp	8	Sender timestamp in milliseconds
18	payload_size	2	Payload length in bytes
20	payload	variable	Command-specific payload

Command header size is 20 bytes.

Table A.7: TCP response frame layout

Offset	Field	Size	Description
0	`magic`	4	ASCII `TRSP`
4	`protocol_version`	1	Current value `0x01`
5	`cmd_id`	2	Echo of command identifier
7	`seq`	2	Echo of command sequence number
9	`result`	1	`0x00 = ACK`, `0x01 = NACK`
10	`err_code`	2	Response error code
12	`payload_size`	2	Optional payload length
14	`payload`	variable	Optional response payload

Response header size is 14 bytes.

A.5 Command Set and Payloads

Table A.8: Command identifier and payload summary

ID	Name	Req. Size	Response Payload
0x0001	CU_STATE	1	none
0x0002	SELFLEV	1	none
0x0003	IBIT	1	none
0x0004	STORAGE	0	none
0x0005	LASER	2	none
0x0006	TURRET	9	none
0x0007	MIRROR	6	none
0x0008	SUT	10	none
0x000A	RESET	0	none
0x000B	DELLOG	0	none
0x000C	FACTORY	0	none
0x000D	SELFTEST	0	none
0x000E	SHUTDOWN	0	none
0x000F	CONFIG	0	active configuration name and runtime path
0x0010	MANUALPOS	32	none
0x0011	POSMODE	1	none
0x0012	REQ_SW_VERSION	0	ASCII software version
0x0013	REQ_HW_INFO	0	ASCII hardware information
0x0014	REQ_WORK_TIME	0	uint32_t uptime_sec

Table A.9: Command payload byte layouts

Command	Field	Offset	Type and Meaning
CU_STATE	cu_state	0	uint8_t requested state ID
SELFLEV	start_stop	0	uint8_t, 1 = start, 0 = stop
IBIT	start_stop	0	uint8_t, 1 = start, 0 = stop
LASER	id	0	uint8_t laser ID from 1 to 8
	on_off	1	uint8_t, 0 = off, 1 = on
TURRET	motion_type	0	uint8_t motion type
	az_data	1	float turret azimuth value
	el_data	5	float turret camera elevation value
MIRROR	mirror_id	0	uint8_t, 1..8 or 0xFF = all
	motion_type	1	uint8_t motion type
	elevation_data	2	float mirror elevation value
SUT	sut_id	0	uint8_t, 1..8 or 0xFF = all
	motion_type	1	uint8_t motion type
	az_data	2	float SUT azimuth value
	el_data	6	float SUT paired mirror value
MANUALPOS	lat	0	double latitude in degrees
	lon	8	double longitude in degrees
	alt	16	double altitude in meters
	heading	24	double heading in degrees
POSMODE	mode	0	uint8_t, 0 = GPS, 1 = Manual

The following request commands have zero-byte request payloads: STORAGE, RESET, DELLOG, FACTORY, SELFTEST, SHUTDOWN, CONFIG, REQ_SW_VERSION, REQ_HW_INFO, and REQ_WORK_TIME.

A.6 Response Payloads

Table A.10: Response payloads for query commands

Command	Payload Type	Meaning
CONFIG	Structured ASCII payload	Active configuration name and runtime path
REQ_SW_VERSION	ASCII bytes	Software version string with no required terminator
REQ_HW_INFO	ASCII bytes	Hardware information string with no required terminator
REQ_WORK_TIME	uint32_t	Uptime in seconds

The `CONFIG` response payload is:

Table A.11: `CONFIG` response payload

Offset	Field	Size	Description
0	`name_len`	1	Length of configuration name
1	`name`	variable	Configuration name as ASCII bytes
`1 + name_len`	`path_len`	2	Configuration path length as `uint16_t`
`3 + name_len`	`path`	variable	Runtime configuration path as ASCII bytes

The default configuration name is expected to be `stc_default.ini`. The path is the exact path used by the running server or `dev/config/stc_default.ini` in local development.

A.7 Command Validation Rules

Table A.12: Command validation rules

Command Group	Validation Rule
All commands	Magic, version, controller role, command ID, payload size, and payload syntax shall be valid
`TURRET`, `MIRROR`, `SUT`	Motion type shall be `0`, `1`, or `2`
`TURRET`, `MIRROR`, `SUT`, `LASER`	Command shall be accepted for execution only when permitted by the state machine
`MIRROR`	Mirror ID shall be `1` to `8`, or `0xFF` for all mirrors
`SUT`	SUT ID shall be `1` to `8`, or `0xFF` for all SUTs
`LASER`	Laser ID shall be `1` to `8`, and `on_off` shall be `0` or `1`
`MANUALPOS`, `POSMODE`	Command shall be accepted only in `SAFE` or `OPER`
Motion values	Values shall be checked against configured axis limits and speed limits

A.8 TLMH Frame

Table A.13: TLMH high-rate telemetry frame

Offset	Field	Size	Description
0	`magic`	4	ASCII `TLMH`
4	`protocol_version`	1	Current value `0x01`
5	`seq`	2	Telemetry sequence number
7	`timestamp`	8	Timestamp in milliseconds
15	`position[18]`	72	Axis positions as `float[18]`
87	`rates[18]`	72	Axis rates as `float[18]`
159	`acceleration[18]`	72	Axis accelerations as `float[18]`

Total TLMH size is 231 bytes.

A.9 Telemetry Axis Mapping

The TLMH arrays and the TLML `motor_status[18]` array use the same canonical axis order.

Table A.14: Telemetry axis and motor-status mapping

Index	**Meaning**
0	Turret azimuth, mast rotation
1	Turret elevation, infrared camera elevation
2	SUT1 azimuth
3	SUT2 azimuth
4	SUT3 azimuth
5	SUT4 azimuth
6	SUT5 azimuth
7	SUT6 azimuth
8	SUT7 azimuth
9	SUT8 azimuth
10	SUT1 elevation or Mirror1 axis
11	SUT2 elevation or Mirror2 axis
12	SUT3 elevation or Mirror3 axis
13	SUT4 elevation or Mirror4 axis
14	SUT5 elevation or Mirror5 axis
15	SUT6 elevation or Mirror6 axis
16	SUT7 elevation or Mirror7 axis
17	SUT8 elevation or Mirror8 axis

A.10 TLML Frame

Table A.15: TLML low-rate telemetry frame

Offset	Field	Size	Description
0	`magic`	4	ASCII `TLML`
4	`protocol_version`	1	Current value `0x01`
5	`controller_id`	1	Control Unit identifier
6	`seq`	2	Telemetry sequence number
8	`timestamp`	8	Timestamp in milliseconds
16	`state`	1	Current STC state
17	`uptime`	4	Uptime in seconds
21	`temperature`	4	Temperature in degrees Celsius
25	`humidity`	4	Relative humidity percentage
29	`pressure`	4	Atmospheric pressure in hPa
33	`position_lat`	8	Latitude in degrees
41	`position_lon`	8	Longitude in degrees
49	`position_alt`	8	Altitude in meters
57	`turret_heading`	8	Heading in degrees
65	`motor_status[18]`	18	One state or health byte per motor axis
83	`laser_status[8]`	8	One on/off status byte per SUT laser
91	`level_x`	4	Platform inclination X
95	`level_y`	4	Platform inclination Y
99	`num_errors`	1	Number of diagnostic codes
100	`error_codes[]`	variable	`num_errors` values of `uint32_t`

The fixed TLML size is 100 bytes. The total size is:

```
100 + 4 * num_errors
```

A.11 Status Arrays

Table A.16: TLML status arrays

Field	Size	Meaning
`motor_status[18]`	18 bytes	One status byte per configured motor axis, using the canonical axis order in Table A.14
`laser_status[8]`	8 bytes	One on/off status byte per SUT laser, where index `i` maps to `SUT(i+1)`

Each motor-status byte gives a compact per-axis health summary in the canonical 18-axis order. The current implementation packs the byte directly from the runtime health fields. The byte records whether the axis is enabled, available, faulted, stale, recently saturated, moving or speed-streaming, or degraded. Saturation is latched for a short observation interval so that brief limit events remain visible in TLML.

Table A.17: Motor-status byte flags

Bit	Mask	Meaning
0	`0x01`	Axis is configured and enabled in the runtime model
1	`0x02`	Axis is available or connected according to the backend snapshot
2	`0x04`	Axis fault is active. For the Nanotec path this includes DS402 fault detection
3	`0x08`	Axis feedback is stale according to the configured timeout
4	`0x10`	Last command for this axis was saturated by the software limit layer
5	`0x20`	Axis is moving or has an active speed-stream command
6	`0x40`	Axis contributes to degraded operation or warning state
7	`0x80`	Reserved for future use

A.12 Diagnostic Code Encoding

Each TLML diagnostic entry is a 32-bit unsigned value.

Table A.18: Diagnostic code structure

Byte Region	Field	Meaning
Byte 3	Category	Top-level subsystem category
Byte 2	Subcategory	Device or subassembly index inside the category
Bytes 1 to 0	Error or warning code	Specific diagnostic code

The diagnostic list is ordered with errors first and warnings next.

Table A.19: Diagnostic category examples

Category	Meaning	Example Subcategories
0	General system	General subsystem
1	Pointing motors	Azimuth motor, elevation motor
2	Mirror elevation motors	Mirror 1, Mirror 2, remaining mirrors
3	SUT azimuth motors	SUT motor 1, SUT motor 2, remaining SUT motors
4	GPS and orientation	GPS receiver, orientation source
5	Self-levelling	Self-levelling actuator or controller
6	Environmental sensors	Sensor 1, additional sensors

A.13 Telemetry Transport Profile

Table A.20: Telemetry transport profile

Stream	Port	Period	Destination
`TLMH`	`6010`	`10 ms`	UDP unicast to active TCP client IP
`TLML`	`6020`	`100 ms`	UDP unicast to active TCP client IP

A.14 Formal Contract Notes

- Each command shall receive one response when the server can reconstruct the response context.

- Response frames echo the received command identifier and sequence number.
- Malformed commands receive a best-effort `NACK` when possible.
- Query commands may return payloads in the response.
- Administrative commands may return an empty `ACK` payload.
- All command payload sizes are exact.
- All motion values are expressed in external-load engineering units.
- Appendix B defines the valid state transitions and state-gate semantics.

Appendix B

State Machine Reference

B.1 Purpose

This appendix defines the STC server state machine as a formal transition reference. It is derived from `stc_state.h`, `stc_state.c`, and the server-side transition effects used by `stc_server`. It also incorporates the operational meaning described in the project state-machine notes and UML.

The main documentation chapters describe why state gating exists. This appendix records the exact transition table, the event semantics, the command-gating rules, and the operational side effects associated with important transitions.

B.2 State Set

The runtime state enumeration is shown in Table B.1. The external design model also contains an `OFF` condition. `OFF` represents the unpowered system condition before the server process starts and is outside the C runtime enumeration.

Table B.1: STC runtime states

Value	State	Operational Meaning
0	INIT	Server startup and initialization phase. Configuration, hardware preparation, CAN setup, and PBIT-related checks belong to this phase.
1	INIT_DONE	Initialization completed. The system is ready to enter a safe commanded state or storage path.
2	SAFE	Powered and controlled standby state. It is the central safe hub for arming, self-levelling, IBIT, and storage entry.
3	OPER	Operational movement state. Turret, SUT, mirror, and speed-stream commands are executable only in this state.
4	SELF_LEVELLING	Self-levelling procedure active. The state exists as the runtime representation of the levelling workflow.
5	IBIT_RUNNING	Interactive Built In Test running. This state is also used as the recovery path from FAIL.
6	STORE	Storage procedure active. Configured axes are commanded toward their storage positions.
7	FAIL	Critical fault state. Motion is inhibited, quick stop is scheduled, faults are latched, and recovery requires diagnostic action.

B.3 Event Semantics Glossary

The state machine is event-driven. Table B.2 defines the operational meaning of each event.

Table B.2: State-machine event glossary

Event	Operational Meaning
EVT_PBIT_OK	Power-on BIT and startup checks completed successfully. Used to leave INIT.
EVT_PBIT_FAIL	Startup checks failed. The system moves to FAIL.
EVT_IBIT_COMPLETE_OK	Interactive BIT completed successfully. The implemented runtime returns to OPER.
EVT_IBIT_COMPLETE_FAIL	Interactive BIT completed with failure. The system moves to FAIL.
EVT_STORE_OK	Storage motion completed successfully. The system returns to SAFE.
EVT_STORE_FAIL	Storage motion failed. The system moves to FAIL.
EVT_ENTER_SAFE	Explicit transition into safe standby. Operationally this clears pending IBIT and self-levelling timers and disables stream control.
EVT_ENTER_OPER	Explicit arming transition from SAFE into operational mode. This enables motion command acceptance at server level.
EVT_SELF_LEV	Start self-levelling from a valid commandable state.
EVT_SELF_LEV_DONE	Self-levelling completed successfully. The implemented runtime returns to OPER.
EVT_IBIT	Start IBIT. This is valid from SAFE, OPER, and FAIL. From FAIL, it is the diagnostic recovery path.
EVT_STORE	Enter storage procedure from a valid state.
EVT_FAULT_CRITICAL	Critical fault detected. This is a global event that moves any current runtime state to FAIL.

B.4 Transition Table

Table B.3 is the normative transition table derived from `stc_state_transition()`.

Table B.3: STC state transition table

Current State	Event	Next State	Type
INIT	EVT_PBIT_OK	INIT_DONE	automatic
INIT	EVT_PBIT_FAIL	FAIL	automatic fault
INIT_DONE	EVT_ENTER_SAFE	SAFE	automatic or commanded
INIT_DONE	EVT_STORE	STORE	commanded
SAFE	EVT_ENTER_OPER	OPER	commanded
SAFE	EVT_SELF_LEV	SELF_LEVELLING	commanded
SAFE	EVT_IBIT	IBIT_RUNNING	commanded
SAFE	EVT_STORE	STORE	commanded
OPER	EVT_ENTER_SAFE	SAFE	commanded
OPER	EVT_SELF_LEV	SELF_LEVELLING	commanded
OPER	EVT_IBIT	IBIT_RUNNING	commanded
OPER	EVT_STORE	STORE	commanded
SELF_LEVELLING	EVT_SELF_LEV_DONE	OPER	automatic completion
SELF_LEVELLING	EVT_ENTER_SAFE	SAFE	commanded abort or safe request
IBIT_RUNNING	EVT_IBIT_COMPLETE_OK	OPER	automatic completion
IBIT_RUNNING	EVT_IBIT_COMPLETE_FAIL	FAIL	automatic fault
IBIT_RUNNING	EVT_ENTER_SAFE	SAFE	commanded abort or safe request
STORE	EVT_STORE_OK	SAFE	automatic completion
STORE	EVT_STORE_FAIL	FAIL	automatic fault
STORE	EVT_ENTER_SAFE	SAFE	commanded safe request
FAIL	EVT_IBIT	IBIT_RUNNING	diagnostic recovery
any runtime state	EVT_FAULT_CRITICAL	FAIL	global fault

Any state and event pair outside Table B.3 is rejected by `stc_state_transition()`.

B.5 Transition Diagram

Figure B.1 illustrates the transition graph.

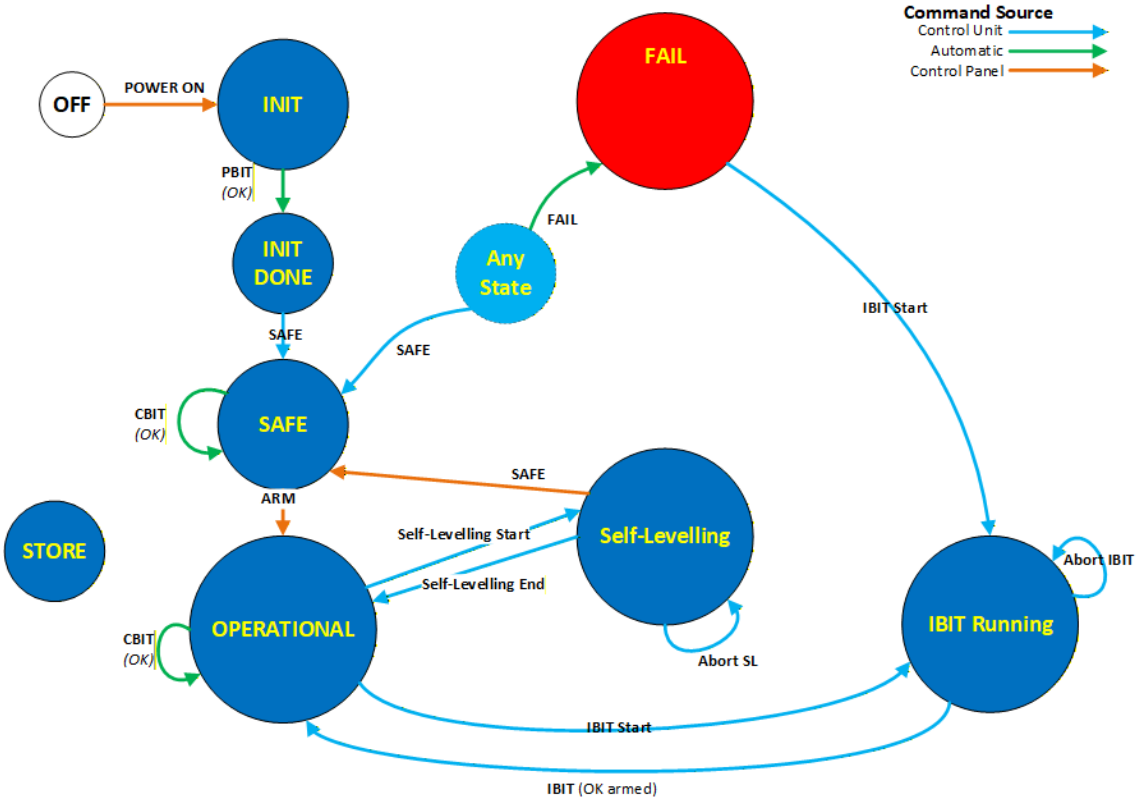


Figure B.1: State Machine Graph.

B.6 Movement and Stream Gating

The state module exposes two execution predicates:

- `stc_state_can_move(state)`
- `stc_state_can_stream(state)`

Both predicates return true only in `OPER`. The runtime command-gating policy is summarized in Table B.4.

Table B.4: Command gating by state

State	Motion Commands	Speed Streaming	Operational Meaning
INIT	blocked	blocked	Startup checks are active
INIT_DONE	blocked	blocked	Ready for safe entry or storage
SAFE	blocked	blocked	Safe standby hub
OPER	allowed	allowed	Normal controlled movement
SELF_LEVELLING	blocked for ordinary motion	blocked	Procedure-owned movement context
IBIT_RUNNING	blocked	blocked	Diagnostic procedure active
STORE	blocked for ordinary motion	blocked	Storage procedure active
FAIL	blocked	blocked	Critical fault condition

The server clears all active stream flags when the resulting state cannot stream. This prevents stale speed commands from persisting after leaving OPER.

B.7 Server-Side Transition Effects

The pure state function computes valid next states. The server adds operational side effects around that transition.

Table B.5: Server-side effects associated with transitions

Condition	Server Effect
Entering `FAIL`	The server sets `fail_stop_pending` and builds `fail_stop_axes_mask`. The mask includes axes that are expected, available, enabled, or actively streaming.
Entering `FAIL` from any previous state	The CAN worker later consumes the pending flag and sends quick-stop commands through the motion backend.
Handling `EVT_ENTER_SAFE`	The server clears pending IBIT and self-levelling timers.
Resulting state cannot stream	The server clears every `axis_stream.enabled` flag.
Starting IBIT	The server marks IBIT as pending and assigns a completion deadline used by the runtime test flow.
Completing IBIT	The server evaluates expected axes, available axes, CAN health, and active fault flags to select the completion event.
Completing self-levelling	The server uses CAN health and self-levelling status to choose the completion event.

These side effects connect the abstract state machine to actual runtime safety behavior. The most important one is the `FAIL` side effect because it converts a software fault state into a hardware-facing quick-stop request.

B.8 Command to Event Mapping

The ICD command layer maps selected commands into state-machine events. The exact mapping depends on payload values and validation result. The operational mapping is shown in Table B.6.

Table B.6: Command to event mapping

Command or Condition	**Generated Event**	**Meaning**
Startup PBIT passed	`EVT_PBIT_OK`	Initialization succeeded
Startup PBIT failed	`EVT_PBIT_FAIL`	Initialization failed
`CU_STATE` requests safe state	`EVT_ENTER_SAFE`	Enter safe standby
`CU_STATE` requests operational state	`EVT_ENTER_OPER`	Arm into operational movement state
`SELFLEV` start	`EVT_SELF_LEV`	Start self-levelling
Self-levelling completion	`EVT_SELF_LEV_DONE`	Complete self-levelling successfully
`IBIT` start	`EVT_IBIT`	Start interactive BIT or recovery BIT
IBIT passed	`EVT_IBIT_COMPLETE_OK`	Complete IBIT successfully
IBIT failed	`EVT_IBIT_COMPLETE_FAIL`	Complete IBIT with failure
`STORAGE`	`EVT_STORE`	Enter storage procedure
Storage completed	`EVT_STORE_OK`	Return to safe state after storage
Storage failed	`EVT_STORE_FAIL`	Enter failure state after storage failure
Critical runtime fault	`EVT_FAULT_CRITICAL`	Enter failure state from any state

B.9 Safety Interpretation

The state machine implements four safety rules.

- **SAFE is the hub state**. It is the neutral powered state used before arming, diagnostics, self-levelling, and storage.
- **OPER is the only ordinary movement state**. All external motion and stream commands are accepted for execution only in `OPER`.
- **FAIL is globally reachable**. A critical fault event overrides the current state and forces `FAIL`.
- **Recovery from FAIL goes through IBIT**. The implemented recovery path is `FAIL` to `IBIT_RUNNING` to `OPER` after a successful test.

This gives the runtime a controlled lifecycle: initialize, enter safe standby, arm for operation, execute motion only while operational, move to failure on critical faults, and require diagnostics before recovery.

Appendix C

Configuration Schema, Axis Mapping, and Examples

C.1 Purpose

This appendix defines the STC runtime configuration schema. The configuration file is an INI file loaded by `stc_server` during startup. It defines network parameters, CAN interface parameters, real-time runtime policy, backend selection, axis scaling, motion limits, functional mappings, SUT mappings, mirror mappings, laser defaults, and diagnostic metadata.

The parser first initializes deterministic defaults, then applies the values found in the configuration file. Optional keys keep their default values when omitted. Malformed sections, unknown keys, invalid values, invalid axis references, missing diagnostic metadata for declared axes, and invalid enabled-axis definitions cause configuration loading to fail.

The main shipped configuration files are:

`dev/config/stc_default.ini`

`dev/config/stc_bench_x8_25.ini`

`dev/config/stc_bench_one_motor.ini`

C.2 INI Syntax Rules

The configuration format follows a simple INI structure.

- Sections are declared as `[SectionName]`.
- Values are declared as `key = value`.
- Empty lines are ignored.
- Lines beginning with the hash character or the semicolon comment character are ignored.
- Inline comments after the hash or semicolon comment characters are ignored.
- Whitespace around section names, keys, and values is trimmed.
- Integer values may use decimal or C-style hexadecimal notation.
- Boolean values accept `1`, `0`, `true`, `false`, `yes`, `no`, `on`, and `off`.
- Unknown sections with active keys are rejected.
- Unknown keys inside known sections are rejected.

The parser is strict by design. A configuration typo is treated as a startup error because an incorrect configuration may change motion limits, node IDs, backend selection, or diagnostic interpretation.

C.3 Accepted Sections

The accepted section families are shown in Table C.1. Axis indices are zero-based. SUT and mirror indices are one-based.

Table C.1: Accepted configuration sections

Section	Requirement	Purpose
`[Network]`	Optional	TCP port, telemetry endpoints, telemetry periods, stale-feedback policy, degraded-startup policy, speed-stream policy, and MYACT target-completion policy
`[CAN]`	Optional	SocketCAN interface name, bitrate, restart delay, discovery enable, and scan range
`[Runtime]`	Optional	Backend selection, RT scheduling policy, RT strictness, thread priorities, controller-ID compatibility, startup feedback grace, degraded-startup policy, speed watchdog, and target-completion timing
`[Backend]`	Optional compatibility section	Legacy backend and single-axis transport defaults. New operational configurations should prefer `[Runtime]` for backend selection and `[CAN]` for CAN interface settings
`[Axis0]` to `[Axis17]`	Required for declared axes	Per-axis enable flag, driver type, node ID, scaling, signs, zeroing policy, limits, storage position, feedback policy, brake policy, active-status polling, and diagnostic metadata
`[Turret]`	Optional	Maps turret azimuth and turret elevation functions to axis indices
`[SUT]`	Optional	Accepts the declarative SUT count field
`[SUT1]` to `[SUT8]`	Optional	Maps each SUT to azimuth, elevation, mirror, and laser defaults
`[Mirror1]` to `[Mirror8]`	Optional	Maps each mirror identifier to an axis index
`[diagnostics]`	Optional	Alternative location for per-axis diagnostic category and subcategory metadata

The value `-1` is accepted in functional mapping fields and means that the logical function is unmapped.

If the same runtime field is assigned more than once, the parser keeps the value parsed last. This is especially important for `backend`, because both `[Runtime].backend` and `[Backend].backend` write to the same runtime backend field. Operational configurations should define the backend selector in one location only.

C.4 Global Configuration Keys

Global configuration keys are grouped by section in Tables C.2, C.3, C.4, and C.5.

C.4.1 `[Network]` keys

Table C.2: `[Network]` keys

Key	Type	Default	Meaning
`tcp_port`	integer	`7000`	TCP command and response listen port
`tlm_dest_ip`	string	`127.0.0.1`	Fallback telemetry destination IP
`tlmh_ip`	string	`127.0.0.1`	Explicit TLMH destination IP
`tlml_ip`	string	`127.0.0.1`	Explicit TLML destination IP
`tlmh_port`	integer	`6010`	TLMH UDP port
`tlml_port`	integer	`6020`	TLML UDP port
`tlmh_interval_ms`	integer	`10`	TLMH period in milliseconds
`tlml_interval_ms`	integer	`100`	TLML period in milliseconds
`tlmh_hz`	double	none	Compatibility key converted to `tlmh_interval_ms`
`tlml_hz`	double	none	Compatibility key converted to `tlml_interval_ms`
`can_scan_max_node`	integer	`0`	Compatibility scan limit for CAN discovery
`can_timeout_ms`	integer	`1000`	Feedback freshness timeout
`strict_stale_policy`	integer	`0`	Enables strict stale-feedback fault policy
`allow_degraded_startup`	bool or integer	`0`	Allows startup to proceed with a degraded hardware inventory when the runtime policy permits it
`require_identity_for_pbit`	bool	`1`	Requires successful identity evidence for PBIT evaluation
`speed_watchdog_ms`	integer	`250`	Speed-stream watchdog timeout
`speed_stream_period_ms`	integer	`50`	Periodic speed-stream reissue cadence
`position_tolerance_deg`	double	`1.0`	Target-completion tolerance used by MYACT motion supervision
`position_timeout_ms`	integer	`5000`	Target-completion timeout used by MYACT motion supervision
`position_poll_ms`	integer	`50`	Target-completion polling period used by MYACT motion supervision

The deprecated keys `mcast_ip`, `mcast_tlmh_port`, and `mcast_tlml_port` are still accepted as compatibility aliases. They generate warnings and map to the current telemetry destination and port fields. New configurations should use `tlm_dest_ip`, `tlmh_port`, and `tlml_port`.

C.4.2 `[CAN]` keys

Table C.3: `[CAN]` keys

Key	Type	Default	Meaning
`ifname`	string	`can0`	SocketCAN interface name
`bitrate`	integer	`500000`	CAN bitrate used by setup and preflight workflows
`restart_ms`	integer	`100`	SocketCAN automatic restart delay
`discovery_enabled`	integer	`0`	Enables discovery behavior when supported by the backend
`scan_max_node`	integer	`0`	Maximum node ID used during discovery

The parser treats `bitrate` as a plain integer and does no unit conversion. A release configuration should use one repository-wide convention for this field.

C.4.3 `[Runtime]` keys

Table C.4: `[Runtime]` keys

Key	Type	Default	Meaning
`backend`	string	build dependent	Selects `can`, `hardware`, `hardware_can`, `sim`, or `simulator`
`rt_enabled`	integer	`1`	Requests real-time scheduling
`rt_strict`	integer	`0`	Makes RT setup failure fatal when set to `1`
`rt_sched_policy`	string	`fifo`	Accepts `fifo`, `rr`, or `other`
`can_worker_priority`	integer	`80`	CAN worker thread priority
`tlmh_tx_priority`	integer	`70`	TLMH telemetry thread priority
`tlml_tx_priority`	integer	`60`	TLML telemetry thread priority
`controller_id_strict_legacy`	integer	`0`	Enables strict legacy controller-ID acceptance
`startup_feedback_grace_ms`	integer	`3000`	Startup grace period before stale feedback can become fault-eligible
`strict_stale_policy`	integer	`0`	Preferred runtime location for stale-feedback fault policy
`allow_degraded_startup`	bool or integer	`0`	Preferred runtime location for degraded-startup policy
`require_identity_for_pbit`	bool	`1`	Preferred runtime location for PBIT identity requirement
`speed_watchdog_ms`	integer	`250`	Preferred runtime location for stream watchdog timeout
`speed_stream_period_ms`	integer	`50`	Preferred runtime location for speed-stream reissue cadence
`position_tolerance_deg`	double	`1.0`	Preferred runtime location for MYACT target tolerance
`position_timeout_ms`	integer	`5000`	Preferred runtime location for MYACT target timeout
`position_poll_ms`	integer	`50`	Preferred runtime location for MYACT target polling period

The key `startup_feedback_grace_ms` is accepted only in `[Runtime]`. The parser rejects the same key when placed in `[Network]` or `[AxisN]`.

C.4.4 [Backend] compatibility keys

Table C.5: [Backend] compatibility keys

Key	Type	Default	Meaning
backend	string	build dependent	Legacy backend selector that writes to the same field as [Runtime].backend
can_if	string	can0	Legacy CAN interface name
node_id	integer	1	Legacy primary node ID
tx_id	integer	0x141	Legacy primary command transmit ID
rx_id	integer	0x241	Legacy primary response receive ID
motion_tx_id	integer	0x401	Legacy motion transmit ID
motion_rx_id	integer	0x501	Legacy motion receive ID
travel_min_deg	double	-140.0	Legacy single-axis minimum travel limit
travel_max_deg	double	140.0	Legacy single-axis maximum travel limit
profile_max_speed_deg_s	double	45.0	Legacy profile maximum speed
profile_max_accel_deg_s2	double	180.0	Legacy profile maximum acceleration
watchdog_timeout_ms	integer	250	Legacy watchdog timeout
telemetry_interval_ms	integer	10	Legacy telemetry interval
experimental_native_speed	integer	0	Legacy experimental speed-mode toggle

The [Backend] section exists for compatibility with earlier single-axis and bench profiles. Integrated deployments should prefer the [Runtime], [CAN], and [AxisN] sections because they describe the current multi-axis runtime model more directly.

C.5 Axis Schema

The runtime supports a fixed maximum of 18 axis slots. The declared axis count is derived from the highest [AxisN] section index found in the file. For example, a file containing [Axis0] to [Axis17] declares 18 logical axis slots.

A declared axis provides metadata. An axis becomes expected by the motion backend only when all of the following are true:

- `enabled = true`
- `driver` is explicitly set to a supported motor family
- `node_id > 0`

Enabled axes are controlled, discovered, supervised, and reported as active runtime participants. Declared axes without `enabled = true` remain part of the logical mapping model and diagnostic schema, although they are not expected by the backend.

Table C.6: `[AxisN]` keys

Key	Type	Meaning
`enabled`	bool	Enables the axis as a runtime-controlled expected axis when the driver and node ID are valid
`driver`	string	Selects the motor protocol family. Accepted aliases include `disabled`, `none`, `pd4c`, `pd4c-can`, `nanotec`, `nanotec_pd4c`, `myact`, `myactuator`, `myactuator-can`, `myact_x8_25`, and `x8_25`
`node_id`	integer	CAN node ID assigned to the axis
`name`	string	Accepted human-readable metadata. The current parser accepts it and does not store it in the runtime configuration structure
`role`	string	Accepted human-readable metadata. The current parser accepts it and does not store it in the runtime configuration structure
`counts_per_rev`	double	Effective drive-side position resolution used for conversion
`gear_ratio`	double	Mechanical ratio between drive-side rotation and controlled-axis motion
`vel_scale`	double	Optional raw-velocity scale used by the axis conversion layer
`vel_raw_mode`	integer	Optional raw-velocity interpretation mode
`min_deg`	double	Minimum allowed external-axis angle
`max_deg`	double	Maximum allowed external-axis angle
`max_speed_deg_s`	double	Maximum allowed external-axis speed
`storage_deg`	double	Storage-position target in degrees
`pp_profile_vel`	`uint16`	Raw DS402 profile-position velocity value
`pp_profile_vel_deg_s`	double	Profile-position velocity in degrees per second. The parser converts it to the raw internal `pp_profile_vel` value using a scale of `0.1 deg/s`
`command_sign`	double	Sign applied to commanded engineering-unit values before backend conversion
`telemetry_sign`	double	Sign applied to feedback values before reporting to the runtime model
`zero_physical_deg`	double	Physical reference angle used by zeroing and MYACT conversion
`zero_raw_deg`	double	Raw actuator angle corresponding to the configured physical zero
`zero_on_start`	bool	Requests software zeroing at backend startup when supported by the selected driver
`brake_release_timeout_ms`	integer	MYACT brake-release timeout for axes that require brake handling

After parsing, every declared axis from `Axis0` through the highest declared axis index must have both diagnostic fields explicitly defined. This rule applies to the declared axis set because TLML diagnostics and motor-status reporting use the canonical 18-slot axis model, while enabled-axis policy controls which axes are expected by the backend.

C.6 Functional Mapping Schema

Functional mappings connect logical turret, SUT, mirror, and laser concepts to axis indices. These mappings are separate from the enabled-axis policy. A mapping can exist for a declared logical model even when a bench profile enables only one physical axis.

Table C.7: Functional mapping keys

Key	Type	Meaning
`[Turret].az_axis`	integer	Axis used for turret mast azimuth, or `-1` when unmapped
`[Turret].el_axis`	integer	Axis used for turret aiming-camera elevation, or `-1` when unmapped
`[SUT].count`	integer	Declarative SUT count accepted by the parser. The runtime recomputes mapped SUT count from valid SUT axis pairs
`[SUTN].az_axis`	integer	Axis used for SUT `N` azimuth, or `-1` when unmapped
`[SUTN].el_axis`	integer	Axis used for SUT `N` elevation, or `-1` when unmapped
`[SUTN].mirror_axis`	integer	Mirror axis associated with SUT `N`, or `-1` when unmapped
`[SUTN].laser_on`	integer	Initial laser state for SUT `N`, where `0` means off and `1` means on
`[MirrorN].axis`	integer	Axis used by mirror `N`, or `-1` when unmapped

A SUT is counted as mapped only when both its azimuth axis and elevation axis are mapped to valid axis indices. A mirror is counted as mapped when its mirror section references a valid axis index.

C.7 Canonical 18-Axis Mapping

The shipped full-turret configuration declares the canonical 18-axis model. In this model, `Axis0` and `Axis1` represent the turret axes, `Axis2` to `Axis9` represent SUT azimuth axes, and `Axis10` to `Axis17` represent SUT elevation and mirror axes.

Table C.8: Canonical axis mapping and default envelope

Axis	Node	Function	Range	Ratio
0	1	Turret azimuth mast	-140 to 140	100
1	2	Turret aiming-camera elevation	-5 to 180	20
2	3	SUT1 azimuth	-35 to 35	20
3	4	SUT2 azimuth	-35 to 35	20
4	5	SUT3 azimuth	-35 to 35	20
5	6	SUT4 azimuth	-35 to 35	20
6	7	SUT5 azimuth	-35 to 35	20
7	8	SUT6 azimuth	-35 to 35	20
8	9	SUT7 azimuth	-35 to 35	20
9	10	SUT8 azimuth	-35 to 35	20
10	11	SUT1 elevation and Mirror1 axis	-12.5 to 18.5	20
11	12	SUT2 elevation and Mirror2 axis	-12.5 to 18.5	20
12	13	SUT3 elevation and Mirror3 axis	-12.5 to 18.5	20
13	14	SUT4 elevation and Mirror4 axis	-12.5 to 18.5	20
14	15	SUT5 elevation and Mirror5 axis	-12.5 to 18.5	20
15	16	SUT6 elevation and Mirror6 axis	-12.5 to 18.5	20
16	17	SUT7 elevation and Mirror7 axis	-12.5 to 18.5	20
17	18	SUT8 elevation and Mirror8 axis	-12.5 to 18.5	20

The shipped full-turret profile uses `max_speed_deg_s = 45` for all canonical axes. It also uses `storage_deg = 0`, `pp_profile_vel = 500`, `diag_cat = 0x06`, and `diag_subcat = 0x3F` across the declared axis set.

C.8 [diagnostics] Section

The [diagnostics] section provides an alternative location for per-axis diagnostic metadata.

Table C.9: [diagnostics] keys

Key	Type	Meaning
axis[N].category	uint8	Diagnostic category for axis N
axis[N].subcategory	uint8	Diagnostic subcategory for axis N

The key motor_status_map is deprecated. The parser accepts it, emits a warning, and ignores it. TLML motor status follows the canonical motor_status[18] axis order defined by the runtime and the ICD appendix.

```
[diagnostics]
axis[0].category = 0x06
axis[0].subcategory = 0x3F
axis[1].category = 0x06
axis[1].subcategory = 0x3F
axis[17].category = 0x06
axis[17].subcategory = 0x3F
```

Listing C.1: Diagnostics metadata example

C.9 Validation Constraints

The loader enforces the constraints in Table C.10.

Table C.10: Configuration validation constraints

Constraint	**Failure Condition**
File access	The configuration file cannot be opened
Section syntax	A section header has no closing bracket
Assignment syntax	A non-comment line is not in `key = value` form
Known sections	A key is found under a section that the parser does not recognize
Known keys	A key is not recognized inside its section
Value parsing	A value cannot be parsed as the required type
Backend selection	`backend` is not one of the accepted hardware-CAN or simulator aliases
Axis driver selection	`driver` is not one of the accepted disabled, Nanotec, or MYACT aliases
Axis section range	`AxisN` uses an index outside `0` to `17`
SUT section range	`SUTN` uses an index outside `1` to `8`
Mirror section range	`MirrorN` uses an index outside `1` to `8`
Functional mapping range	Turret, SUT, or mirror mappings use an axis value other than `-1` or a valid declared axis index
Startup grace placement	`startup_feedback_grace_ms` appears outside `[Runtime]`
Diagnostic metadata	Any declared axis from `Axis0` through the highest declared axis lacks diagnostic category or subcategory
Enabled-axis driver	An enabled axis has no explicit driver or has a disabled driver
Enabled-axis node ID	An enabled axis has `node_id <= 0`
Enabled-axis limits	An enabled axis has `min_deg > max_deg`
Active-status polling	An axis enables `active_status2_enable` with `active_status2_interval_ms <= 0`
Stream watchdog	`speed_watchdog_ms < 0`
Stream period	`speed_stream_period_ms <= 0`
Position tolerance	`position_tolerance_deg <= 0`
Position timeout	`position_timeout_ms <= 0`
Position poll period	`position_poll_ms <= 0`
Startup feedback grace	`startup_feedback_grace_ms < 0`

The parser validates syntax, supported keys, core ranges, enabled-axis readiness, and timing-policy sanity. Engineering review and commissioning should also verify the following physical constraints:

- `counts_per_rev > 0` for axes that use encoder-count conversion

- `gear_ratio > 0` for geared axes
- `max_speed_deg_s > 0`
- `storage_deg` lies inside `[min_deg, max_deg]`
- node IDs are unique on the same CAN bus
- enabled axes use a driver compatible with the connected device
- MYACT axes define the zeroing and brake policy required by the mechanical setup
- functional mappings reference enabled axes in operational deployments
- backend selection is defined in one location only

C.10 Failure Behavior

If configuration loading fails, `load_config()` returns `false`. The loader logs the file path, line number, section name, key name, and failure reason when available. Configuration failure is fatal for a correctly deployed runtime because the server cannot safely infer axis limits, node IDs, driver types, feedback policy, and diagnostic routing from an invalid file.

The main failure categories are:

- file access failure
- syntax failure
- schema failure
- type failure
- range failure
- placement failure
- diagnostic metadata failure
- enabled-axis readiness failure

C.11 Canonical Full-Turret Configuration Excerpt

The following excerpt captures the structure of the shipped full-turret configuration. It is abbreviated to avoid repeating all identical axis blocks.

```
[Network]
tcp_port = 7000
tlm_dest_ip = 127.0.0.1
tlmh_port = 6010
tlml_port = 6020
tlmh_interval_ms = 10
tlml_interval_ms = 100
can_timeout_ms = 1000
strict_stale_policy = 0

[Runtime]
backend = can
startup_feedback_grace_ms = 3000

[Axis0]
enabled = true
driver = pd4c
node_id = 1
counts_per_rev = 4096
gear_ratio = 100
min_deg = -140
max_deg = 140
max_speed_deg_s = 45
storage_deg = 0
pp_profile_vel = 500
diag_cat = 0x06
diag_subcat = 0x3F

[Axis1]
node_id = 2
counts_per_rev = 4096
gear_ratio = 20
min_deg = -5
max_deg = 180
max_speed_deg_s = 45
storage_deg = 0
pp_profile_vel = 500
```

```
diag_cat = 0x06
diag_subcat = 0x3F

[Axis2] to [Axis9]
node_id = 3 to 10
counts_per_rev = 4096
gear_ratio = 20
min_deg = -35
max_deg = 35
max_speed_deg_s = 45
storage_deg = 0
pp_profile_vel = 500
diag_cat = 0x06
diag_subcat = 0x3F

[Axis10] to [Axis17]
node_id = 11 to 18
counts_per_rev = 4096
gear_ratio = 20
min_deg = -12.5
max_deg = 18.5
max_speed_deg_s = 45
storage_deg = 0
pp_profile_vel = 500
diag_cat = 0x06
diag_subcat = 0x3F

[Turret]
az_axis = 0
el_axis = 1

[SUT1] to [SUT8]
az_axis = 2 to 9
el_axis = 10 to 17
laser_on = 0

[Mirror1] to [Mirror8]
axis = 10 to 17

[diagnostics]
axis[0].category = 0x06
axis[0].subcategory = 0x3F
...
```

```
axis[17].category = 0x06
axis[17].subcategory = 0x3F
```

Listing C.2: Full-turret configuration structure

The shipped full-turret file also contains a `[Backend]` compatibility section. Since backend selection is parsed in file order, final deployment files should keep a single backend selector to avoid ambiguous startup behavior.

C.12 One-Motor Bench Configuration Excerpt

The one-motor bench profile is a deterministic laboratory configuration shared by the server, GUI, CLI tools, and telemetry receiver. It keeps the higher-level interface unchanged while allowing the hardware driver to be swapped between `myact_x8_25` and `pd4c`.

```
[Network]
tcp_port = 7000
tlmh_ip = 127.0.0.1
tlmh_port = 6010
tlmh_hz = 100
tlml_ip = 127.0.0.1
tlml_port = 6020
tlml_hz = 10
can_scan_max_node = 1

[CAN]
ifname = can0
bitrate = 1000000
restart_ms = 100

[Runtime]
backend = can
rt_enabled = 0
rt_strict = 0
strict_stale_policy = 0
startup_feedback_grace_ms = 3000
speed_watchdog_ms = 2000
speed_stream_period_ms = 50

[Axis0]
enabled = true
name = bench_one_motor
```

```
driver = myact_x8_25
node_id = 1
role = mirror1
counts_per_rev = 36000
gear_ratio = 9
min_deg = -90.0
max_deg = 90.0
max_speed_deg_s = 50.0
storage_deg = 0.0
command_sign = 1.0
telemetry_sign = 1.0
zero_on_start = true
zero_physical_deg = 0.0
brake_release_timeout_ms = 3000
brake_poll_ms = 50
active_status2_enable = true
active_status2_interval_ms = 50
feedback_timeout_ms = 1000
pp_profile_vel_deg_s = 50
diag_cat = 0x06
diag_subcat = 0x3F

[Turret]
az_axis = -1
el_axis = -1

[Mirror1]
axis = 0

[SUT1]
az_axis = -1
el_axis = -1

[diagnostics]
```

Listing C.3: One-motor bench configuration excerpt

This profile is useful for validating command routing, telemetry publication, stream watchdog behavior, target completion, GUI integration, and motor-family switching with the smallest safe hardware setup.

C.13 Recommended Release Configuration Rules

The following rules should be applied before freezing a release configuration.

- Define the backend selector in one section only.
- Use `[Runtime].backend` for new deployments.
- Use `[CAN].ifname`, `[CAN].bitrate`, and `[CAN].restart_ms` for SocketCAN setup.
- Keep `[Backend]` only for compatibility profiles that still need the legacy single-axis fields.
- Give every declared axis diagnostic metadata.
- Give every enabled axis an explicit supported driver and a positive node ID.
- Keep bench profiles deliberately small and explicit.
- Keep full-turret profiles explicit about all 18 logical axis slots.
- Remove deprecated `motor_status_map` from new files.
- Use a consistent CAN bitrate unit convention across all shipped profiles.

These rules keep the configuration file auditable and prevent hidden behavior changes during deployment, commissioning, and hardware substitution.

Appendix D

CANopen and DS402 Mapping

D.1 Purpose

This appendix defines the CANopen and CiA 402 mapping assumed by the Nanotec motion path. It focuses only on the objects, identifiers, PDO payloads, and state-machine behavior used by the implementation and by the commissioning utilities.

The same mapping is used for the Nanotec PD4-C motor used during laboratory development and for the Nanotec PD6-C family used for the turret azimuth axis. Both drives expose the CANopen object dictionary and CiA 402 power-drive profile used by the low-level `pd4c` library.

D.2 CiA 402 Conceptual Basis

CiA 402 defines the behavioral profile for the drive. The drive is controlled through a finite state automaton. State transitions are requested through object `6040h` Controlword, while the current state is reported through object `6041h` Statusword.

The implementation uses this model directly:

- `6040h` is sent in RPDO1 for real-time state control.
- `6041h` is received in TPDO1 for state observation.
- `6060h` selects the requested operating mode.
- `6061h` reports the active operating mode.

- Profile Position mode is used for absolute position moves.
- Velocity mode is used for streamed speed control through `6042h`.

The server does not model the drive as a generic motor. It models the drive as a DS402 device with explicit state, mode, fault, and stop semantics.

D.3 CANopen Identifier Mapping

For a drive with CANopen node ID `N`, the Nanotec library uses the identifiers shown in Table D.1.

Table D.1: CANopen identifiers used by the Nanotec path

Service	**COB-ID**	**Purpose**
NMT	`0x000`	Start, stop, pre-operational, and reset commands
SDO request	`0x600 + N`	Expedited object-dictionary read and write request
SDO response	`0x580 + N`	Expedited object-dictionary response or abort code
RPDO1	`0x200 + N`	Runtime controlword and operating mode command
RPDO2	`0x300 + N`	Profile-position target and profile velocity
RPDO3	`0x400 + N`	Velocity-mode target velocity
TPDO1	`0x180 + N`	Runtime statusword and mode display feedback
TPDO2	`0x280 + N`	Position feedback
TPDO3	`0x380 + N`	Velocity feedback
Heartbeat	`0x700 + N`	CANopen node-liveness state
EMCY	`0x080 + N`	Emergency object from the drive

The low-level library installs CAN RAW filters for SDO responses, TPDO1, TPDO2, TPDO3, heartbeat, and EMCY. Other frames are ignored by the library and can be inspected with `candump` during commissioning.

D.4 PDO Payload Layouts

The runtime mapping is fixed and intentionally compact. All multi-byte values are little-endian.

Table D.2: Receive PDO payloads sent from STC to the drive

PDO	DLC	Bytes	Mapped Object
RPDO1	3	0..1	6040h Controlword, uint16
RPDO1	3	2	6060h Modes of Operation, int8
RPDO2	6	0..3	607Ah Target Position, int32
RPDO2	6	4..5	6081h Profile Velocity, uint16
RPDO3	2	0..1	6042h Target Velocity, int16

Table D.3: Transmit PDO payloads received from the drive

PDO	DLC	Bytes	Mapped Object
TPDO1	3	0..1	6041h Statusword, uint16
TPDO1	3	2	6061h Modes of Operation Display, int8
TPDO2	4	0..3	6064h Position Actual Value, int32
TPDO3	2 or 4	0..1 or 0..3	6044h Velocity Actual Value, parsed as int16 for 2-byte frames and int32 for 4-byte frames

The main real-time rule is simple: PDOs carry runtime control and feedback, while SDOs are reserved for setup, diagnostics, commissioning, and non-periodic reads.

D.5 PDO Object-Dictionary Mapping Entries

The drive object dictionary must contain the PDO mappings in Table D.4. These are the mappings expected by the code.

Table D.4: PDO communication and mapping objects

PDO	OD Objects	Expected Mapping
RPDO1	1400h, 1600h	6040h:00h length 16 and 6060h:00h length 8
RPDO2	1401h, 1601h	607Ah:00h length 32 and 6081h:00h length 16
RPDO3	1402h, 1602h	6042h:00h length 16
TPDO1	1800h, 1A00h	6041h:00h length 16 and 6061h:00h length 8
TPDO2	1801h, 1A01h	6064h:00h length 32
TPDO3	1802h, 1A02h	6044h:00h length 16 or 32 according to the active drive mapping

During PDO remapping, the drive should be placed in a non-operational commissioning condition. The safe procedure is:

1. Disable the PDO through its communication parameter.
2. Set the mapping object subindex 00h to zero.
3. Write the mapped entries.
4. Set subindex 00h to the final number of mapped objects.
5. Re-enable the PDO through the communication parameter.
6. Verify the mapping through SDO reads.
7. Save only after the mapping is confirmed on the bench.

Persistent saving uses 1010h Store Parameters with the Nanotec save signature 65766173h. The exact save subindex depends on the device family and object category, so it must be selected from the corresponding PD4-C or PD6-C manual during commissioning.

D.6 Object Dictionary Entries Used by the Implementation

Table D.5 lists the runtime objects that are actively used by the Nanotec library and CAN backend.

Table D.5: Runtime object dictionary entries

Object	**Name**	**Implementation Use**
6040h	Controlword	DS402 state transitions, quick stop, fault reset, profile-position setpoint handshake
6041h	Statusword	DS402 state decode, fault-bit detection, target-reached and setpoint-ack observation
6060h	Modes of Operation	Runtime mode request through RPDO1
6061h	Modes of Operation Display	Active mode verification through TPDO1 or SDO
607Ah	Target Position	Profile Position target through RPDO2
6081h	Profile Velocity	Profile Position velocity through RPDO2
6042h	Target Velocity	Velocity-mode speed command through RPDO3
6064h	Position Actual Value	Position feedback through TPDO2 and diagnostics
6044h	Velocity Actual Value	Velocity feedback through TPDO3 and diagnostics
603Fh	Error Code	Diagnostic SDO read during bench tests
1001h	Error Register	Diagnostic SDO read during bench tests
1018h	Identity Object	Node discovery and identity verification
2009h	CANopen Node ID	Node-ID cross-check during discovery when supported

Table D.6 lists the additional objects used by the standalone calibrated motion demo and commissioning workflow.

Table D.6: Commissioning and calibration object dictionary entries

Object	**Name**	**Use**
2300h	NanoJ Control	Stop NanoJ before direct motion tests when it appears active
2301h	NanoJ Status	Check whether NanoJ is running
608Fh	Position Encoder Resolution	Read active drive units per motor revolution for raw scaling
6091h	Gear Ratio	Read drive-internal gear ratio for comparison with the mechanical model
6092h	Feed Constant	Read drive unit-stack context for logging
60A8h	SI Unit Position	Log configured position unit context
60A9h	SI Unit Velocity	Log configured velocity unit context
607Ch	Home Offset	Log home-offset context
607Dh	Software Position Limit	Optionally widen bench software limits so the calibrated $\pm 140°$ range fits
607Fh	Max Profile Velocity	Optional startup write for profile-speed ceiling when supported
6080h	Max Motor Speed	Optional startup write for motor-speed ceiling
6083h	Profile Acceleration	Optional startup write for acceleration profile
6084h	Profile Deceleration	Optional startup write for deceleration profile
6085h	Quick Stop Deceleration	Optional startup write for quick-stop ramp
6086h	Motion Profile Type	Optional startup write for trapezoidal or jerk-limited profile selection
60C5h	Max Acceleration	Optional startup write for acceleration ceiling
60C6h	Max Deceleration	Optional startup write for deceleration ceiling
1601h:02h	RPDO2 Mapping Entry	Read mapping length to detect whether 6081h is mapped as 16 or 32 bits

The commissioning utilities keep unit-stack writes disabled by default. The preferred strategy is to read the active drive scaling, apply the additional mechanical ratio in software, and modify persistent drive objects only when

the change is deliberate and verified.

D.7 DS402 Controlword Sequences

The implementation uses the controlword sequences in Table D.7. The operating mode byte is sent together with the controlword in RPDO1.

Table D.7: Controlword sequences used by the implementation

Action	**Controlword**	**Meaning**
Shutdown	`0x0006`	Request Ready to Switch On
Switch on	`0x0007`	Request Switched On
Enable operation	`0x000F`	Request Operation Enabled
Quick stop	`0x000B`	Clear quick-stop bit while keeping enable-related bits set
Fault reset pulse	`0x0080` then `0x0000`	Pulse bit 7 to request fault reset
Profile Position setpoint start	`0x001F`	Raise new-setpoint bit 4 while operation remains enabled
Profile Position setpoint release	`0x000F`	Lower new-setpoint bit after the edge is accepted

For Profile Position moves, STC sends `607Ah` and `6081h` through RPDO2, then toggles bit 4 through RPDO1. For relative moves, bit 6 is added by the low-level move function. The server normally uses absolute moves at axis level after converting degrees into drive counts.

D.8 Statusword Interpretation

The implementation decodes the DS402 state with mask `0x006F`. The interpreted patterns are shown in Table D.8.

Table D.8: Statusword state patterns used by the implementation

Masked Value	State	Use
0x0000	Not Ready to Switch On	Startup or unavailable drive state
0x0040	Switch On Disabled	Expected state after boot or reset
0x0021	Ready to Switch On	Intermediate enable sequence state
0x0023	Switched On	Intermediate enable sequence state
0x0027	Operation Enabled	Required state for motion execution
0x0007	Quick Stop Active	Stop state after quick-stop request
0x000F	Fault Reaction Active	Fault reaction in progress
0x0008	Fault	Fault state requiring reset and recovery

The statusword bits in Table D.9 are used directly or treated as commissioning indicators.

Table D.9: Relevant statusword bits

Bit	Mask	Meaning in STC
0	0x0001	Ready to switch on state component
1	0x0002	Switched on state component
2	0x0004	Operation enabled state component
3	0x0008	Fault bit used by `stc_faults` and server health logic
5	0x0020	Quick-stop state component
6	0x0040	Switch-on-disabled state component
10	0x0400	Target reached indication used during commissioning and move observation
12	0x1000	Setpoint acknowledge used by Profile Position handshake

D.9 Mode Mapping

Only the modes in Table D.10 are used in the active STC control path.

Table D.10: Operating modes used by STC

Value	Mode	Use
1	Profile Position	Absolute axis moves through 607Ah and 6081h
2	Velocity	Streamed speed commands through 6042h
3	Profile Velocity	Defined in the library enum for completeness, although the current streamed-speed path uses Velocity mode 2 because RPDO3 maps to 6042h

The backend enables the requested mode before sending motion commands. If the axis is already enabled in the same mode, the backend reuses the current DS402 state.

D.10 EMCY and Heartbeat Policy

The library receives and stores both EMCY and heartbeat frames.

Table D.11: EMCY and heartbeat handling

Signal	Stored Data	Policy
EMCY	error code, error register, five manufacturer bytes, timestamp	Treated as critical evidence for the affected drive. It is logged and exposed through the backend snapshot.
Heartbeat	node state byte and timestamp	Treated as liveness context. It supports diagnostics and availability assessment.
TPDO freshness	latest TPDO timestamp per axis	Used as the primary stale-feedback indicator.

The safety implication is intentional. EMCY means the drive explicitly reported an emergency condition, so it supports transition into FAIL or degraded behavior depending on the affected axis. Heartbeat absence alone

is not fatal in the default policy because boot timing, reset transitions, and temporary network-management states can interrupt heartbeat. Stale TPDO feedback and DS402 fault bits are stronger runtime indicators for motion supervision.

D.11 Commissioning Checklist

A Nanotec axis should pass the following checks before being used by the STC server:

1. Confirm node ID and baud rate using the device switches or object dictionary settings.
2. Confirm SocketCAN can transmit and receive on the selected interface.
3. Read `6041h` through SDO and verify a valid DS402 statusword response.
4. Start the node with NMT.
5. Read `1018h` identity subobjects and record vendor, product, revision, and serial.
6. Cross-check `2009h` node ID when supported.
7. Verify RPDO and TPDO mappings through `1400h..1402h`, `1600h..1602h`, `1800h..1802h`, and `1A00h..1A02h`.
8. Verify mode display through `6061h` after requesting mode `1` or `2`.
9. Verify position feedback through TPDO2 or `6064h`.
10. Verify velocity feedback through TPDO3 or `6044h`.
11. Verify quick stop from a low-speed bounded test.
12. Verify fault reset only after the device-side cause has been understood.
13. Save persistent communication or mapping changes only after a complete bench verification.

This checklist keeps drive commissioning aligned with the runtime assumptions of the server, the CAN backend, and the low-level Nanotec library.